MSc in Computer Science – Team Project

Interim Report

Group 3 – ClauseGuard

Understand your lease before you agree: AI-assisted compliance checking for Irish residential leases



Team Members

A00047237 – Siddharth Varma Raja

A00047190 – Prasad Nitin Pansare

A00047193 – Shubham Tatyasaheb Patil

A00046404 – Kameshwara Karthik Ayyalasomayajula

A00046791 – Jude Fernandes

GitHub Repo Link: https://github.com/Siddharth-Varma-Raja/ClauseGuard

22/10/2026

Technological University Dublin · School of Computer Science

Table of Contents

1. Introduction 4

2. User Scenario 5

2.1 Identifying the Target Users 5
2.2 Personas 6
2.3 Collective Insights 7
2.4 User Journeys and User Flows 8
2.5 Why They Are Important 9
2.6 What Problem We Solve for Them 9

3. Technical Problem 10

3.1 Why the System Is Needed 10
3.2 Core Technical Problem 10
3.3 The Law as a Moving Target 11
3.4 Worked Example 12
3.5 Review of Existing Solutions 12
3.6 Research Background 14

4. Data 16

4.1 Information Sources 16
4.2 Legal Source Register 16
4.3 Lease Data Collection and Labelling 16
4.4 Cleaning and Pre-processing 18
4.5 Application Data and Storage 18
4.6 Data Pipeline 19

5. Technical Solution 21

5.1 System Functionality 21
5.2 Architecture and Technology Stack 22
5.3 Design Process: From Sketches to the Built Interface 23
5.4 Front End 26
5.5 Back End 29
5.6 Model Experiments 31
5.7 Deployment and DevOps 32
5.8 Testing and Quality 32
5.9 Key Design Decisions 32

6. Evaluation 34

6.1 Success Criteria 34
6.2 Evaluation Results So Far 34
6.3 Technical Evaluation Methodology 39
6.4 User Evaluation Methodology 40
6.5 Legal Correctness Check 41
6.6 How Evaluation Feeds Development 42

7. Conclusion: The Plan 43

7.1 Project Management Strategy 43
7.2 Team Roles and Contributions 43
7.3 Meetings, Retrospectives and Team Health 44
7.4 Progress to Date 45

7.5 Response to Supervisor Feedback 46
7.6 Current Challenges and Risks 46
7.7 Timeline and Deliverables 47
7.8 Summary 47
8. References and Key Resources 48

1. Introduction

Renting is the main form of housing for a large and growing share of people in Ireland, yet the document that governs a tenancy, the residential lease, is written in legal language that most tenants cannot check. A lease is usually signed under time pressure: rooms are scarce, landlords and agents expect a quick answer, and the tenant rarely has a solicitor at the signing table. Clauses that conflict with Irish tenancy law or with Residential Tenancies Board (RTB) guidance therefore pass unnoticed: a deposit of two months' rent, a landlord who may “enter at any time”, a rent review every six months, or a notice period shorter than the statute allows.

ClauseGuard is our answer to that gap. It is a privacy-preserving web application in which a tenant uploads a lease (PDF or photo), and receives an overall verdict, every clause colour-coded, and, for each clause that needs attention, a plain-English explanation of why, what the tenant can do next, and the legal source the check is based on. Around this core the system offers a **notice checker** for letters a tenant receives from a landlord (termination, rent increase, entry), a **tenancy timeline** for rent dates and deadlines, an **Ask ClauseGuard** assistant for questions about a specific clause, and settings for explanation detail, data export and deletion.

The idea went through several rounds before it was fixed. It was first proposed on 20 September 2026; after our first mentor meeting the team considered dropping it for a new idea, and on 30 September, after advice from our mentor, we decided to stay with ClauseGuard and improve it significantly rather than start again. The two strongest pieces of advice that shaped the product since then were: (1) the test of the product is whether an ordinary person with no legal background understands the output, so lawyers are not the right people to judge the interface; and (2) highlighting alone is not enough, the tenant needs a verdict, a reason and a next step.

The project has a research dimension as well as an engineering one. Beyond building the tool, we will evaluate *how best to build it*, by comparing a keyword/rule baseline, a zero-shot language model, a fine-tuned classifier and a retrieval-augmented hybrid on a purpose-built, frozen test set of labelled lease clauses, and by measuring whether real tenants understand their lease better after using ClauseGuard (System Usability Scale and a comprehension quiz).

This report follows the structure used for the module: the users and why they matter (Section 2), the technical problem and existing solutions (Section 3), the data we rely on (Section 4), the technical solution as built (Section 5), how we evaluate it and what we have measured so far (Section 6), and our project management, challenges and plan for the remaining weeks (Section 7). Throughout, we state clearly which parts work today and which are still being built.

ClauseGuard is an educational project and does not give legal advice. It checks leases against published RTB guidance and Irish tenancy law as we understand them, and it may be wrong or out of date. For advice about a specific situation, tenants should contact the RTB, Threshold or a solicitor.

2. User Scenario

2.1 Identifying the Target Users

ClauseGuard is for tenants in the private rental sector in Ireland who are about to or have recently signed a tenancy who don't know the law. In the Explore phase we created a mind map of those affected (Figure 1), and reduced those groups to those who are most vulnerable to an unfair lease and not at all able to check one:

- Students, including international students – with a first-time renting need, often under deadline to take the room before the next student, lack of knowledge of the rules and sometimes reading the books in a second language.
- Recent immigrants to Ireland, both international and skilled, who are not familiar with the normal clauses in Ireland.
- Last-minute workers – those who have limited time to read through a contract, or who are unable to make an appointment with an advice service, who may only have access to a telephone.
- Older tenants and tenants with disabilities – for whom readable text, resizable fonts, and a simple flow is important.
- Advice services (secondary users) – organisations like Threshold that currently look through the leases manually, could use ClauseGuard as a tool.

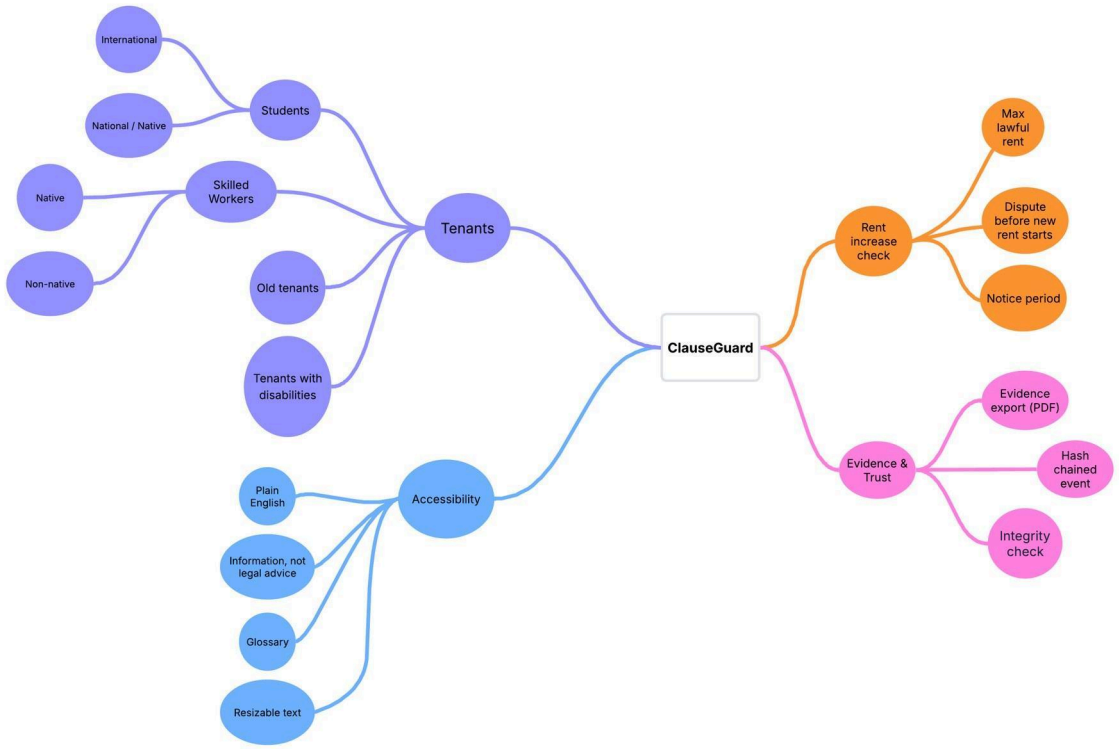


Figure 1: *ClauseGuard mind map: tenant groups, accessibility needs, and the two core value strands (rent-increase checking; evidence and trust).*

2.2 Personas

We generated two personas from the mind map and initial conversations we had with leasing tenants we contacted as we collected the leases: a student who was signing his/her first lease under pressure, and a busy worker about a rent increase on his/her cell phone. Persona 1 – Pavan Kumar Polasam, international student (25, Dublin 7)

How ClauseGuard is helpful: In a few clicks, the upload returns a verdict ("2 clauses are worth checking") and the deposit clause is marked with guidance for the 1 month period and a suggested question for the landlord, as well as a downloadable report.

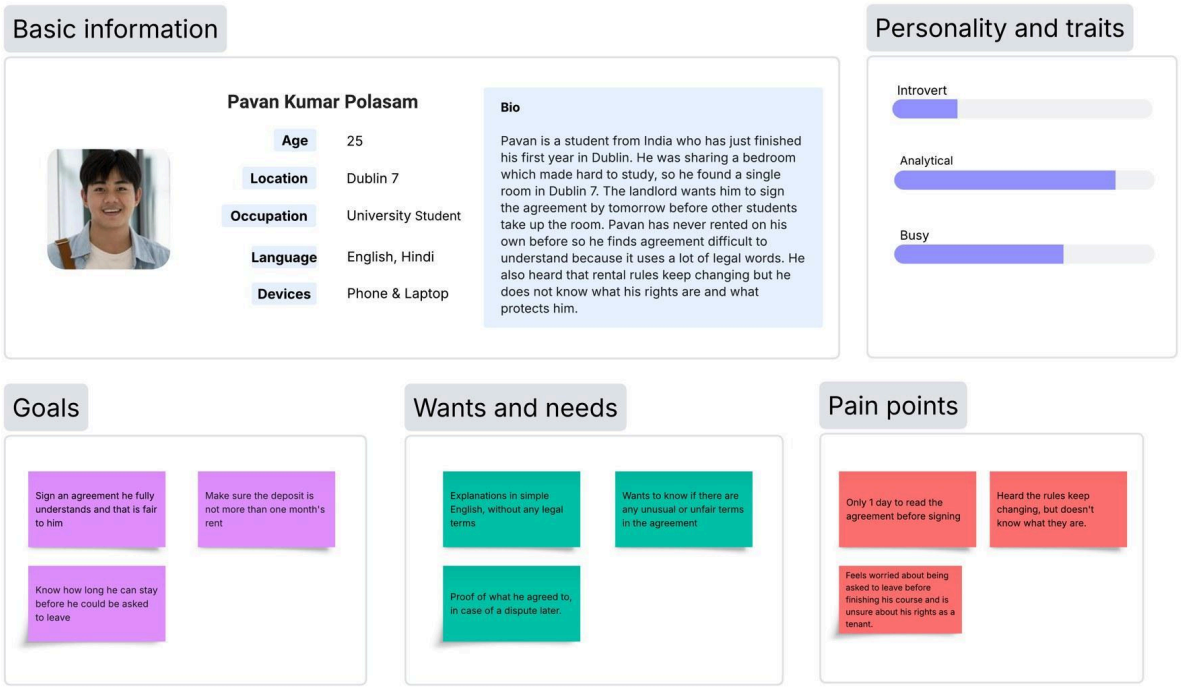


Figure 2: *Persona 1 – Pavan, university student renting alone for the first time.*

Persona 2 – Isabel Flores, part-time worker (45, Dublin 13)

Last year, the rent of Isabel's landlord went up and he tells her that it will go up from €950 to €1050 next month. She gets advice from a friend about the rules for rent increases, but she is not familiar with the rules or when the new rent would begin. Only a cell phone is her only tool.

How ClauseGuard helps: The notice checker will pose a few simple questions about the letter and provide a verdict and legal source, as well as next steps, for each check performed; The timeline will show the date the new rent would go into effect and the deadline for referring a dispute to the RTB. You can see the checker's actual output for Isabel's letter in section 6.2.

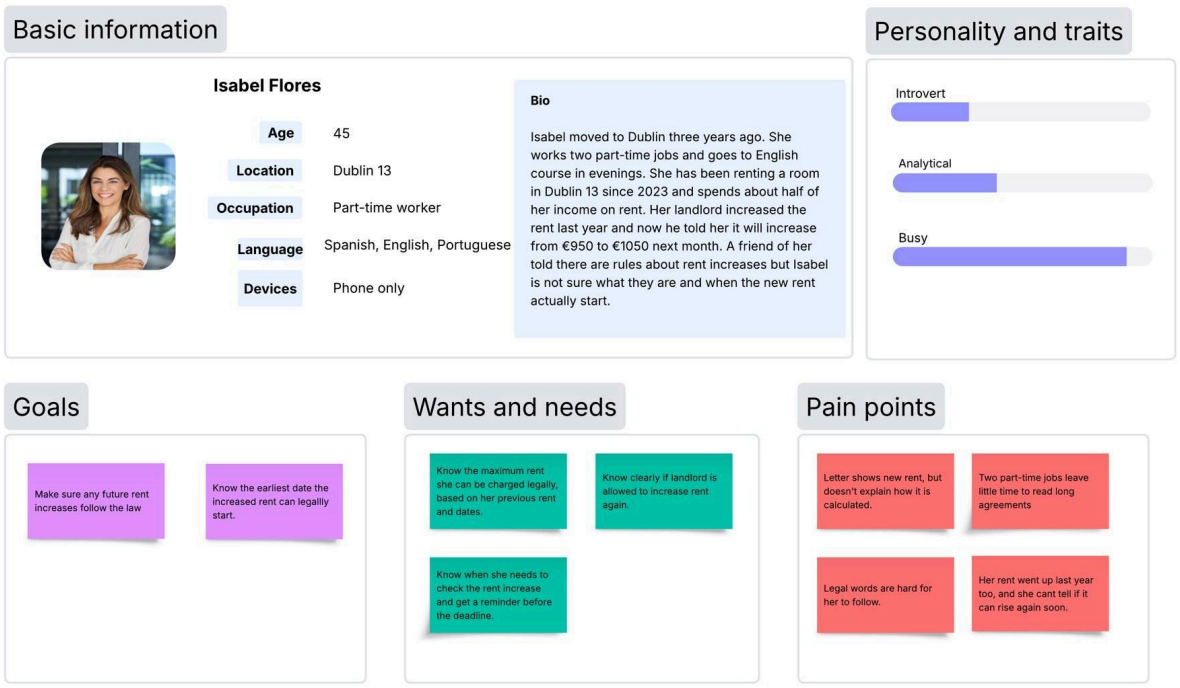


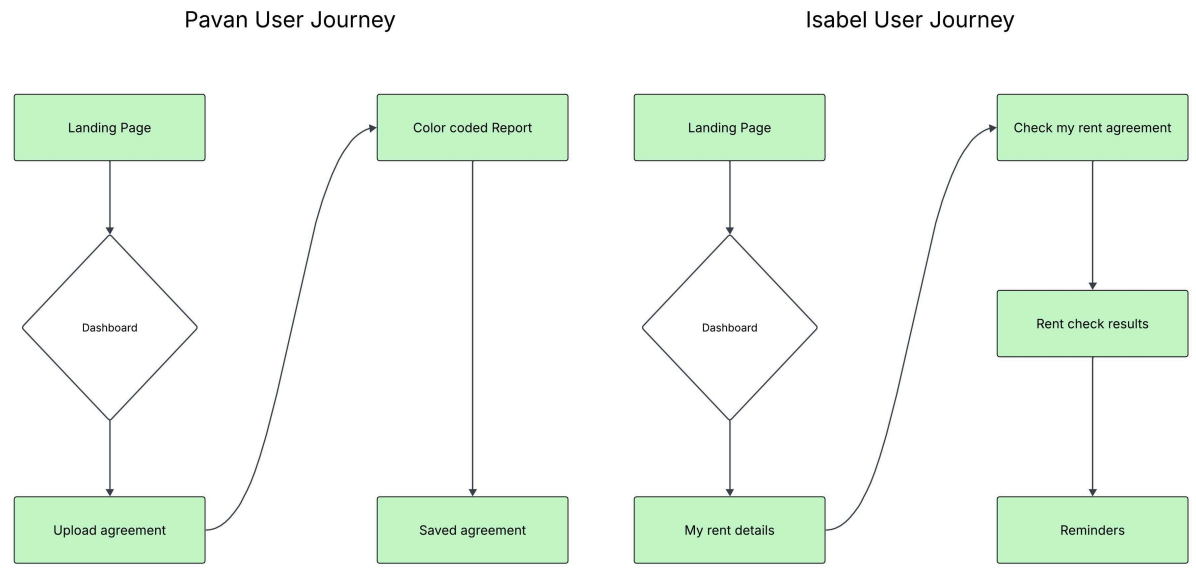
Figure 3: *Persona 2 – Isabel, part-time worker facing a rent increase.*

2.3 Collective Insights

- While they are from different age groups, speak different language, there are some overlaps in the needs that guide the priorities of the product.
- The time pressure is the norm. They don't have days, they have hours. The user is not forced to go any further until he or she wishes to.
- Citations should be avoided in favor of plain English. Both are reluctant to read the law. The explanation and the following step are the most important, but legal sources must be available.
- Privacy is a precondition. The names, addresses, PPS numbers and bank details are included on a lease. Only one will be uploaded to users if personal data is removed prior to anything being stored or sent to an external service.

2.4 User Journeys and User Flows

For each persona we mapped a high-level journey (the pages they visit) and a detailed flow with decision points.



Pavan: landing → dashboard → upload → colour-coded report → saved agreement

Isabel: landing → dashboard → my rent details → check rent letter → results → reminders

Figure 4: *High-level user journeys for the two personas.*

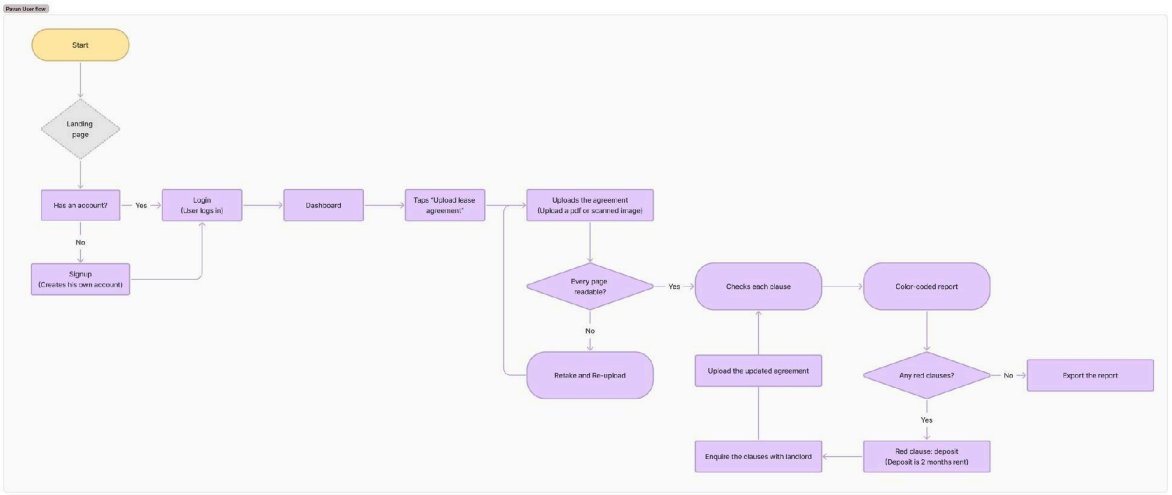


Figure 5: *Detailed user flow for Pavan: account, upload, readability check, clause checks, red-clause handling and export.*

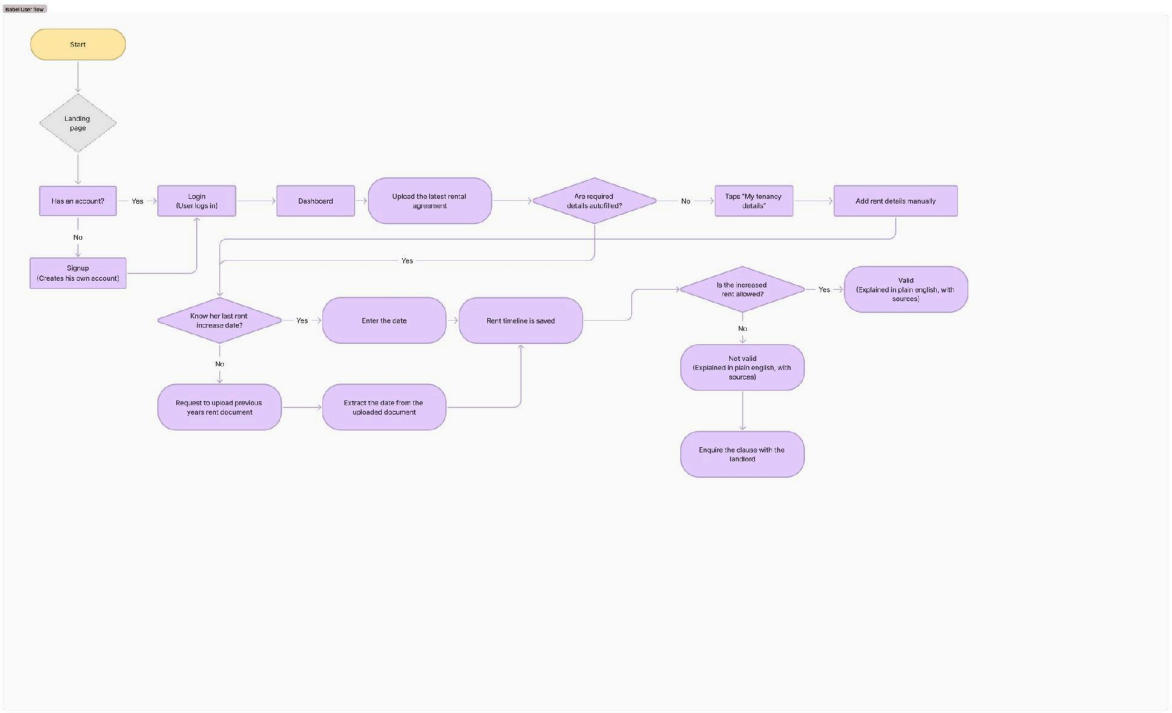


Figure 6: *Detailed user flow for Isabel: checking a rent-increase letter and setting a reminder.*

2.5 Why They Are Important

A lease is one of the most important documents a tenant signs, as it determines how much they pay and what their rights are. However, with housing in short supply, tenants often have little choice but to accept the terms offered. This can lead to problems such as unfair rent increases, short notice periods, excessive deposits or hidden fees, especially for first-time renters.

Changes to Irish tenancy laws have made it even harder for tenants to understand their rights. The rules can vary depending on when a tenancy started and when a notice was served. ClauseGuard aims to make this easier by helping tenants understand their leases and identify potential issues before they become bigger problems.

Ultimately, the goal is to help tenants make informed decisions, avoid unnecessary disputes and feel more confident about their rights.

2.6 What Problem We Solve for Them

Tenants often struggle to know whether their lease is fair and follows the law. Reading legal documents can be confusing, online searches can be a headache, and professional advice may not be available when they need it.

ClauseGuard makes this process easier by allowing tenants to upload their lease and receiving a report. It highlights clauses in different colours based on their risk level, explains why they may be a concern, and provides steps with links to relevant legal sources. Tenants can also ask questions about specific clauses and download a report to share with their landlord.

3. Technical Problem

3.1 Why the System Is Needed

Tenants need to understand their contract at the one moment it matters, before they sign, but the information they need is buried in a long lease and in a body of law that changed twice in 2026. The RTB publishes guidance, forms, a rent calculator and a dispute service, but it does not read an individual tenant's lease. Advice services such as Threshold do read leases, but with limited capacity and waiting times. General-purpose AI chatbots will summarise a pasted lease instantly, but they may invent law, give no verifiable source, and receive the tenant's unredacted personal data. ClauseGuard exists to combine the authority of the official sources, the speed of software and a privacy-first design, in a format built for people with no legal background.

3.2 Core Technical Problem

Stated in one sentence, the core problem is: **produce a trustworthy, explainable, date-correct compliance judgement for every clause of an unstructured, privacy-sensitive document, for a reader who is not a lawyer.** Each part of that sentence is a separate technical difficulty, and each maps onto a component in Section 5.

Table 2: *The core technical problem broken into sub-problems, why each is hard, and our approach.*

Sub-problem	Why it is hard	Our approach
Ingestion	Leases arrive as born-digital PDFs, scanned PDFs, phone photos and Word files. Text order, headers, footers and page numbers break naive extraction; clause numbering styles vary (1., 1.1, (a), (iv), bullets, schedules).	pdfplumber per page for textual pdf and Tesseract OCR for image to extract text; a clause splitter that recognises numbering and heading patterns, strips footers and drops signature blocks and headings.
Is this even a lease?	Users upload the wrong file (invoices, notices, IDs). Analysing a non-lease wastes resources and produces nonsense.	Template diff engine scores tenancy vocabulary (10 core + 10 secondary terms) and rejects uploads that do not look like a residential tenancy agreement (HTTP 422).
Privacy	Leases contain names, addresses, Eircodes, PPS numbers, phone numbers, emails and IBANs. The data must not be stored or sent to an external AI provider.	Redaction before storage: Irish-specific regular expressions plus spaCy named-entity recognition; only redacted text reaches S3; the original file and raw text stay in memory until processed
Clause understanding	Legal language is ambiguous and domain-specific. We have very little labelled Irish lease data, so a learned classifier cannot yet be shown to beat a trivial baseline.	Nine-topic taxonomy; keyword baseline as a stand-in; labelling tool and lease-level splits; fine-tuning experiments with Legal-BERT and Irish LegalBERT (Section 5.6).
Compliance reasoning	Whether a deposit is too high or a notice too short is arithmetic on extracted numbers and dates. A language model is the wrong tool: it is not repeatable and cannot show its working.	Deterministic rule engine. Rules are data (YAML) with a citation, a basis (law vs guidance), plain-English templates and test cases.
The law moves	The answer depends on the tenancy start date (before or after 1 March 2026) and on the notice service date (before or after 14 September 2026).	Versioned legal source register with a point-in-time map; rules carry `applies_from` / `applies_to`; unknown date → needs review.

Sub-problem	Why it is hard	Our approach
Trust and uncertainty	A confident wrong answer is the worst outcome for a tenant.	The engine never guesses: low classifier confidence, missing values, no verified rule or no start date all return “needs review”. Law and RTB guidance are shown differently.
Explanation	Highlighting alone does not help a non-expert, and statute text is unreadable for most users.	Verdict banner, colour-coded clauses on the page, and for each clause: what it means, why it was flagged, what you can do, and sources.
Tampering and evidence	A landlord may change a document after signing; PDF metadata can be edited.	SHA-256 hash of the uploaded bytes stored with the record as a tamper-evident fingerprint.
Untrusted input	A lease is attacker-controlled text that will reach a language model (Ask ClauseGuard). Hidden text or injected instructions could manipulate the answer.	Only the selected clause and the question are sent; system instruction fixed; planned: hidden-text detection and an injection test set with attack-success rate.

Two of these problems deserve particular emphasis because they drive the architecture. First, **compliance decisions must be deterministic and traceable**. If a tenant asks “why is this red?”, the answer must be a rule that a person can read, with the number that was extracted and the source it came from. This is why we separated *understanding* (which clause is about what, a task where machine learning helps) from *deciding* (whether the clause breaks a rule, a task for written rules). Second, **the system must know when it does not know**. Research on legal AI tools has shown that large language models produce confident but false legal statements at significant rates (Dahl et al., 2024; Magesh et al., 2025). Our design therefore treats abstention (“needs review”) as a feature, not a failure.

3.3 The Law as a Moving Target

Figure 7 shows the legislative events that the knowledge base must represent. The two 2026 boundaries are the most important: rules on security of tenure, termination grounds and initial rent-setting split on whether the **tenancy started** before or after 1 March 2026, while rules on how notices are served and copied to the RTB split on when the **notice was served** (before 1 March 2026, between 1 March and 13 September 2026, or from 14 September 2026). The rent cap applies to both groups.

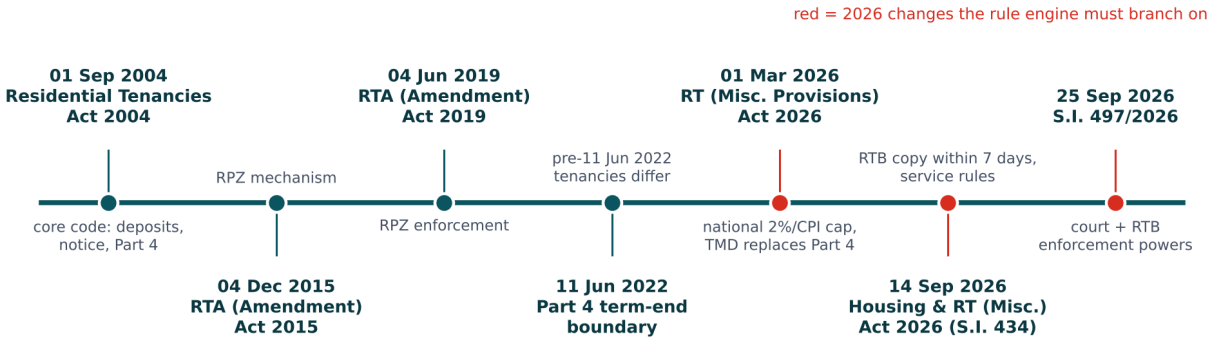


Figure 7: *Legislative timeline encoded in the legal source register (red: 2026 changes that the rule engine must branch on). Dates as recorded in the register (v2, 9 October 2026).*

Table 3: *Extract from the point-in-time map in the legal source register: how the applicable rule depends on dates.*

Rule area	Tenancy started before 1 Mar 2026	Tenancy started on/after 1 Mar 2026	Resolver logic
Security of tenure	Part 4 tenancy, unlimited duration after 6 months	Tenancy of Minimum Duration (TMD), rolling 6-year cycles after 6 months	Branch on lease start date
Rent cap	Was RPZ-based (HICP or 2%) inside an RPZ	National cap: lower of 2% or CPI, once a year, everywhere	Pre-2026: check RPZ status at review date
Termination on sale	Generally allowed, subject to Part 4 notice rules	Large landlords (4+ tenancies or a company): no sale ground; small landlords: limited grounds	Start date AND landlord size
Market-rent reset	Not allowed unless substantially refurbished	Only at tenancy start (limited cases) or end of a 6-year TMD cycle	Start date; refurbishment flag
Notice service and RTB copy	Personal or postal service	From 14 Sep 2026: RTB copy within 7 days; termination notices not by post	Branch on notice *service* date
Dispute deadlines	10 / 21 calendar days	From 14 Sep 2026: 10 working days	Count working days

3.4 Worked Example

Table 4 follows one clause through the pipeline. It is the kind of clear-cut, numeric check our supervisors asked us to get right first. We show both the *target* behaviour and what the system does *today*, because the gap between the two is exactly the work planned for the next sprint.

Table 4: *Worked example: one rent-review clause through the ClauseGuard pipeline (target behaviour versus today).*

Stage	Target behaviour	Today (22 October)
Input	"The Landlord may review and increase the rent every 6 months, based on market value."	Same clause, clause 6 of our synthetic test lease.
Extraction and redaction	Text extracted from the PDF (OCR if scanned); names, addresses and Eircodes replaced with markers.	Works. Note: spaCy also replaces the word "Landlord" with a marker (Section 6.2), which must be fixed.
Split and classify	Clause 6, topic *rent review*, confidence reported.	Split correctly; topic *rent review* guessed by the keyword baseline.
Rule check	Rent may be reviewed at most once a year and increases are capped at the lower of 2% or CPI (register P-01, P-02). A six-monthly review conflicts with P-01 → red, breaks the law.	No lease rule for rent review yet, so the clause is grey: not assessed (honest, but not useful). The notice checker already implements the 12-month rule; it must be moved into `rules.yaml`.
Output	Plain-English reason: "Rent can only be reviewed once a year, and any increase is limited to 2% or inflation, whichever is lower." Next step: ask for the clause to say "once a year". Sources: RTA 2004 as amended by the 2026 Act; RTB "Rental law changes from 1 March".	"This looks like a 'rent review' clause. We do not have a checked rule for this topic yet, so it has not been assessed."

3.5 Review of Existing Solutions

We reviewed the options a tenant in Ireland realistically has today, using the competitor analysis board in Figure 8 and a wider look at general AI tools and enterprise contract-review products. All are useful in their own way; none gives a tenant a quick, private, source-backed check of *their own* lease at the moment of signing.



Figure 8: *Competitor analysis board: location, type, pricing, audience and core functionality of four existing services.*

Residential Tenancies Board (rtb.ie)

The RTB is the public body for the rental sector. Its website is free and authoritative: plain-language guidance, dispute applications, the tenancy register and a rent calculator for permitted increases. It is the source our rules are checked against. Its limitation for our users is that it gives *general* information: it does not read a tenant's lease, and a first-time renter has to know which page applies to which clause, and which version of the rules applies to their tenancy dates.

Threshold and Citizens Information

Threshold provides free, trusted advice to tenants, including a phone helpline and advisers who will look at a lease or a notice; Citizens Information publishes clear explanations of renting rights. Both are excellent, but human advice is a scarce resource with waiting times and limited hours, and it is rarely available at the signing table late in the evening. We see these services as partners, not competitors: ClauseGuard's report is designed to be something a tenant can bring to an adviser, and advice services are a secondary user group.

Rentview

Rentview is a property-management platform for Ireland and the UK, priced from about €25 to €299 per month, aimed at agents and landlords for tenant and lease management. It solves the landlord's administrative problem, not the tenant's understanding problem.

Legallens and general-purpose AI assistants

Legallens is a global AI service for tenants (about €7.99 per month) offering AI risk analysis of rental agreements. It is the closest in concept, but it is not built around Irish tenancy law or its 2026 changes, and, like general chatbots, it relies on a language model to make the judgement. General-purpose assistants can summarise a pasted lease and answer questions quickly, but they may hallucinate legal rules, rarely cite a verifiable Irish source, do not know which version of the law applies to the tenancy dates, and receive the tenant's unredacted personal data.

Enterprise contract-review AI (for example Kira, Luminance)

Commercial contract-analysis platforms are powerful clause extractors used by law firms and corporate legal teams. They are priced and designed for lawyers reviewing commercial contracts, not for an individual tenant, and they are not configured for Irish residential tenancy rules or plain-English explanation.

Table 5: *Comparison matrix of existing options and ClauseGuard (✔ full support, ◐ partial, ✘ none; ClauseGuard marked as of today, with planned items noted).*

Capability	RTB site	Threshold / CI	Rentview	General AI chat	Legallens	ClauseGuard
Reads the tenant's own lease	✘	◐ (adviser)	✘	✔	✔	✔
Based on Irish tenancy law and RTB guidance	✔	✔	◐	✘	✘	✔ (rules draft)
Knows which law applies by date	◐	✔	✘	✘	✘	◐ (engine ready; dates not extracted)
Shows the legal source for each finding	✔	◐	✘	✘	◐	✔
Says "not sure" instead of guessing	n/a	✔	n/a	✘	✘	✔
Removes personal data before AI processing	n/a	n/a	✘	✘	?	✔
Checks landlord notices	◐ (calculator)	✔	✘	◐	✘	✔
Available instantly, free	✔	✘	✘	✔	✘	✔
Plain English for non-lawyers	✔	✔	✘	◐	◐	✔

ClauseGuard's position is therefore to combine the authority of the RTB sources, the speed of an automated assistant and a privacy-first design, delivered in a format for people with no legal background. Its distinctive technical choices, a deterministic rule engine with explicit abstention and date-versioned rules, are the features the AI-first alternatives lack.

3.6 Research Background

Our approach draws on several strands of research. **Legal language models**: BERT (Devlin et al., 2019) and its legal adaptation LEGAL-BERT (Chalkidis et al., 2020) showed that pre-training on legal text improves performance on legal tasks; we tested this for Irish law (Section 5.6). **Clause datasets**: LEDGAR (Tuggener et al., 2020) provides over a million labelled contract provisions and CUAD (Hendrycks et al., 2021) provides expert-annotated clauses for contract review; neither covers Irish residential leases, which is why we must build our own labelled set. **Unfair-clause detection**: CLAUDETTE (Lippi et al., 2019) detected potentially unfair clauses in online terms of service, a close analogue to our task in consumer law. **Grounding**: retrieval-augmented generation (Lewis et al., 2020) grounds answers in retrieved documents and is the planned architecture for clauses that rules cannot decide. **Reliability**: studies of legal hallucination (Dahl et al., 2024; Magesh et al., 2025) and of indirect prompt injection in LLM-integrated applications (Greshake et al., 2023) motivate our abstention and input-isolation design; calibration (Guo et al., 2017) and conformal prediction (Angelopoulos and Bates, 2021) give principled

ways to decide when to abstain. **Evaluation**: inter-annotator agreement with Cohen's kappa (Cohen, 1960) and the System Usability Scale (Brooke, 1996) with its commonly used benchmark of 68 (Bangor et al., 2008; Sauro, 2011).

Our supervisors advised us not to choose an accuracy target ourselves but to find the standard used in research on lease and contract interpretation and keep improving towards it. A focused literature review on accuracy benchmarks, text simplification and the visualisation of legal information is therefore part of the plan for the coming week (Section 7.7).

4. Data

4.1 Information Sources

ClauseGuard relies on three types of data: Legal knowledge (what the law and RTB guidance says and when), Lease data (real and synthetic lease clauses to develop and analyse the clause analysis) and Application data (what users upload and the results that we store for them). Both have different quality, privacy and freshness needs.

Table 6: *Data sources used by ClauseGuard, their purpose and their status.*

Source	Content	Used for	Status
Legal source register v2 (team-compiled)	18 Acts and statutory instruments; 27 clause-relevant provisions; 12 RTB compliance checks; 11-row point-in-time map; change log	Seeding rules and their citations; deciding which rule applies by date	Done (v2, 9 Oct); 4 provisions open or partial
Irish Statute Book; rtb.ie guidance pages	Statute text and plain-language guidance	Verifying each rule before it is marked verified	Ongoing
Irish Acts corpus (500 Acts) and RTA 2004	Legislative text	Domain pre-training of Legal-BERT (experiment 1)	Done (experiment)
LEDGAR sample (10,000 clauses)	Labelled contract provisions (SEC filings)	Additional pre-training text	Done (experiment)
Leases from a public Google Form and contacts	Real and template leases, redacted	Clause splitting, labelling, test set	Partial (3 leases, 289 clauses labelled).
Synthetic and adversarial clauses	Clauses written to cover each rule and edge case	Rule test cases; future test set	Partial (11 rule cases, 2 test leases)
Application data (MySQL, S3)	Users, documents, reviews, notices, events, preferences; redacted text	Running the product	Done (schema 0001–0004)

4.2 Legal Source Register

The legal source register is the first human-readable form of the project's knowledge base, in the legal sense. All provisions are reviewed against the RTB's own published guidance (and not simply law-firm commentaries) and all changes are recorded with source and whether or not a legal reviewer should review. There are six sheets in the workbook:

- **Legislation register (18 rows):** every Act, amending Act and statutory instrument in scope, with year, type, status, commencement date and what superseded it. Superseded instruments (for example the 2008 and 2017 Housing Standards Regulations) are marked so they are never cited as current.
- **Provision register (27 rows, P-01 to P-27):** one row per clause-relevant rule, mapped to a category, its statutory source, the RTB guidance page, and which tenancies it applies to (pre-2026, post-2026, both, or a specific date).
- **RTB compliance check (12 checks):** for example, does the register reflect that RPZs were abolished rather than extended, and does it apply the 2% / CPI cap to both new and existing tenancies?

- **Point-in-time map (11 rule areas):** the rules that diverge by date, with the resolver logic for each (summarised in Table 3).
- **Change log (10 entries)** and **read-me / maintenance guidance:** re-check the RTB-verified column each sprint, and after any new Act or RTB update.

Table 7: *Selected provisions from the register and how they feed ClauseGuard (RTB-verified status as recorded on 9 October 2026).*

ID	Category	Rule (summary)	RTB verified	Used by
P-01	Rent review frequency	Rent can be reviewed at most once per year.	Yes	Notice checker; next lease rule
P-02	Rent increase cap	Increase capped at the lower of 2% or CPI inflation.	Yes	Notice checker (to align)
P-04 / P-05	Security of tenure	Part 4 (pre-1 Mar 2026) versus Tenancy of Minimum Duration (post-1 Mar 2026).	Yes	Termination rules
P-06 / P-07	Termination by landlord size	Large landlords (4+ tenancies or a company) cannot end a TMD to sell; small landlords have limited grounds.	Yes	Planned rule; needs landlord size (“not sure” allowed)
P-08	RTB copy of notices	Copy to the RTB within 7 days for notices served from 14 Sep 2026 (previously same day).	Yes (changed 14 Sep)	Notice checker
P-13	Deposit protection	Deposit lodged with a statutory scheme.	Open – high risk	Deliberately NOT encoded until confirmed
P-14	Minimum housing standards	Rented homes must meet the 2019 Regulations.	Partial	Repairs clauses (planned)
P-25	Rent-setting notice copy	One-month window to copy rent-setting notice to the RTB.	Open – commencement unconfirmed	Not encoded

4.3 The collection and labelling of leases.

The team is creating its own real Irish lease to share as an open data set, as there is no such thing as an open set of real Irish leases. 30 September onwards leases have been gathered via a community shared public google form and personalised contact.

Labeling is done with a small command line tool (language_stream/label_tool.py), which displays one clause at a time and notes a topic from a limited taxonomy of 9 categories: deposit, notice period, rent review, termination, repairs, access, fees, waiver and other. The tool is resumable, that is, clauses are not re-labelled in the subsequent run, and split_by_lease is used to always split the train and test sets by lease, which prevents clauses coming from the same lease being present in both sets - this would overestimate the results.

This labelling scheme for the evaluation is more than topic. For every clause we will note: clause boundaries; category and extracted values (amounts, periods, dates); verdict (compliant, possible problem, non-compliant, needs review) based on the law in force on the date of the tenancy; and the supporting Act section. Labeling will proceed according to a written guide, based on a pilot round and then will be performed by a second annotator, with adjudication, and agreement will be calculated using

Cohen's kappa, before the test set will be frozen on 15 November. The goal is to have at least 600 - 800 clauses (real, synthetic and adversarial).

4.4 Cleaning and Pre-processing

Raw extracted text is noisy. The pre-processing chain applied before any analysis is:

1. **Per-page extraction.** pdfplumber returns the text per page; pdf2image, Poppler render the page to pdf, then text is read by Tesseract OCR (300dpi). Currently the pages are concatenated with newlines so each clause is reported as page 1, but the analysis service is already aware of format feeds so switching the extractor to them is a minor fix that will be implemented in the coming sprint.
2. **Redaction.** Regular expressions for IBAN, PPS number, Irish phone numbers, email, Eircode, Dublin postal districts and passport numbers, then spaCy NER for person names (PERSON) and places (GPE, LOC, FAC).
3. **Clause splitting.** A new clause starts at numbered patterns (1., 1.1, 2), (a), (iv)), bullets, or headings such as SPECIAL CONDITIONS, FIRST SCHEDULE and SIGNED.
4. **Footer removal and whitespace normalisation**, for example page footers like “1 | Page” and “Page | 3 of 9”.
5. **Junk filtering.** Fragments under 40 characters, all-capitals headings under 100 characters, and short signature or witness blocks are dropped. On lease 2 this reduced 120 split fragments to 110 clauses.
6. **Value extraction.** Euro amounts, “two months' rent” phrases, notice lengths in days/weeks/months and entry notice in hours; each function returns *nothing* rather than a guess when unsure.

4.5 Application Data and Storage

All the application data is persisted in MySQL 8 using SQLAlchemy 2 (async, aiomysql) with Alembic migrations and redacted lease text saved in AWS S3. There are four migrations that have created ten tables. All primary keys are UUIDv4 strings, so they cannot be enumerated; all timestamps are UTC; and DELETE /api/me is a cascading delete of all rows and stored files associated with the user.

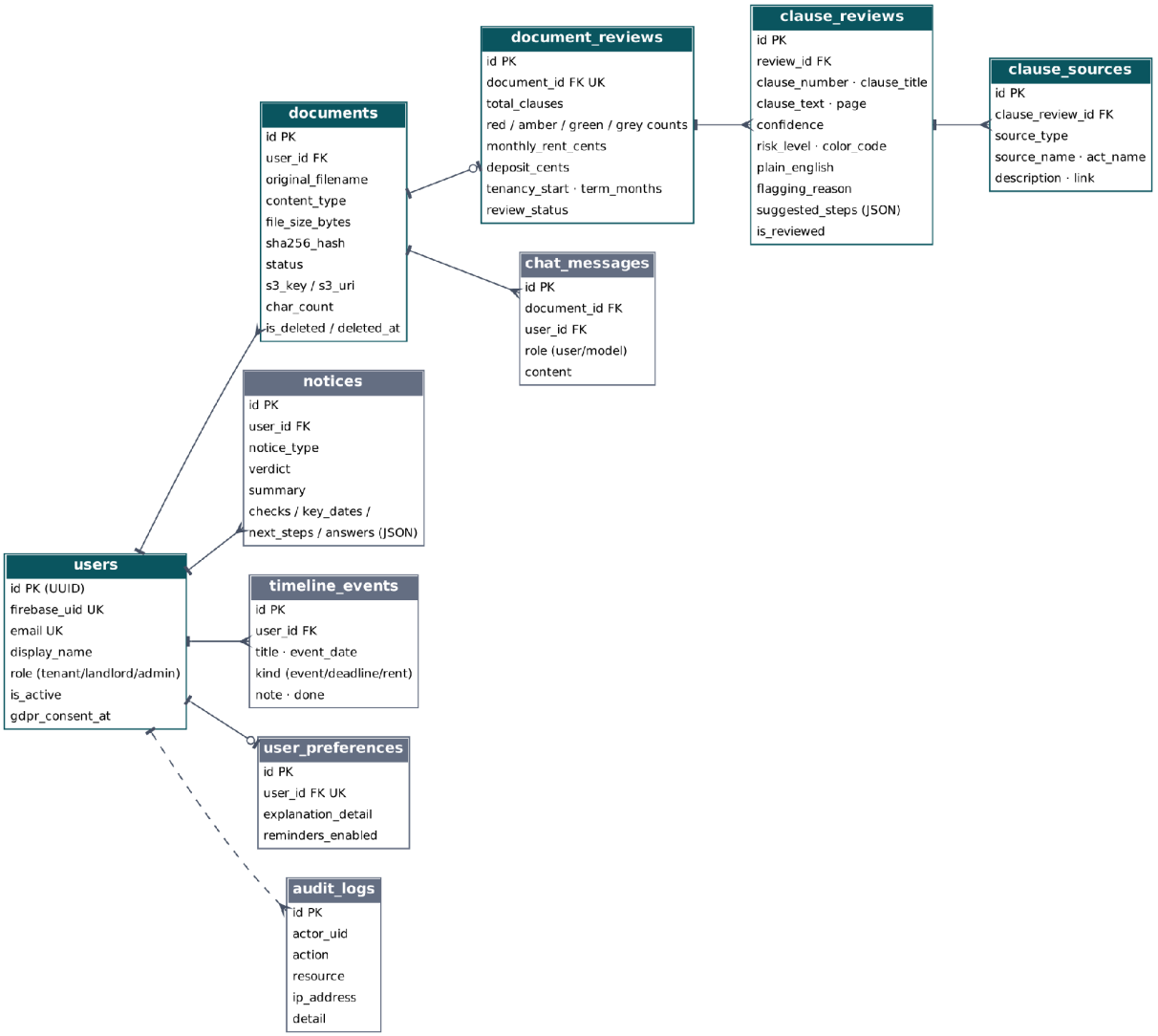


Figure 9: *Entity-relationship diagram of the ClauseGuard database (teal: lease-analysis tables; grey: supporting tables).*

Table 8: *Database migrations applied so far.*

Migration	File	Creates
0001	0001_initial_schema.py	`users`, `audit_logs`
0002	0002_add_documents_table.py	`documents` with SHA-256 hash and S3 references
0003	0003_add_chat_and_review_tables.py	`document_reviews`, `clause_reviews`, `clause_sources`, `chat_messages`
0004	0004_add_notices_timeline_preferences.py	`notices`, `timeline_events`, `user_preferences`; structured lease terms; clause page and confidence

4.6 Privacy, Retention and Ethics

Privacy is designed in, not added on. Table 9 lists what is stored, where, and for how long.

Table 9: *What ClauseGuard stores, where, and how it is removed.*

Data item	Where	Handling
Original uploaded file	Nowhere	Processed in memory only; never written to disk or S3.

Data item	Where	Handling
Raw (unredacted) text	Nowhere	Exists only in memory during the request.
SHA-256 hash, file name, size, type	MySQL `documents`	Tamper-evident fingerprint and metadata; deleted with the account.
Redacted lease text	S3 private bucket	Key `documents/<user>/<uuid>.txt`; purged by delete-all and account deletion; lifecycle expiry planned.
Clause review results	MySQL	Clause text (from redacted text), verdict, explanation, sources; cascade-deleted.
Questions to Ask ClauseGuard	Browser + Gemini API	Only the selected clause, the question and the conversation so far are sent to Gemini; answers are not stored server-side (a `chat_messages` table is ready for later).
Account and audit log	Firebase + MySQL	Email, display name, consent time; audit events (register, logout, password-reset request) with IP address.

This is an open trade-off with our supervisors: Ask ClauseGuard is passed an redacted version of the text of the clauses to an external service (Google Gemini). We restrict it to the selected clause and not the entire lease, and the quality problem of redaction outlined in Section 6.2 must be addressed before inviting real users. This includes a short Data Protection Impact Assessment (DPIA), with ethics approval to be requested prior to the use of actual leases of participants in the labelled test set.

5. Technical Solution

5.1 System Functionality

ClauseGuard is a web app written in Python. A tenant registers with email and password, uploads a lease (PDF or image up to 25MB), and after processing, sees a verdict banner (for example “2 clauses are worth checking”), counts of red, amber, green and grey clauses, a rendered page-by-page version of the lease in which each clause is highlighted with the explanation of the clause in plain English, reasons for each flag, what to do, and the sources on the side panel. The tenant can take through the clauses, check each one by checking it, can ask a question regarding it, after which the tenant can download a text report. The notice checker walks the tenant through a termination, rent-review or entry notice and returns a verdict with individual checks, key dates and next steps, separately; the timeline displays rent dates and deadlines; and settings allow for a simple or detailed explanation, an export of all data, or the deletion of all data.

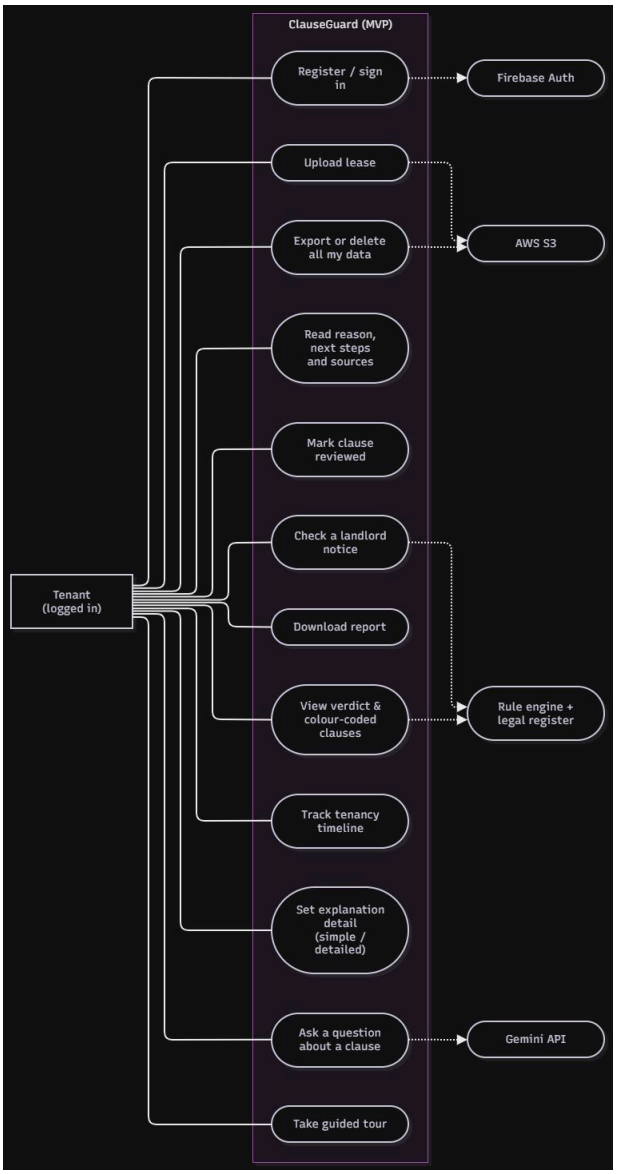


Figure 10: *High-level use-case diagram of the MVP for a logged-in tenant*

The main use case relationships are: view verdict implies upload lease (a review is only available for an uploaded document); read reason, next steps and sources extends view verdict; ask a question is supported by the clause selected in a review; check a notice is along with the rule engine; export or delete all my data implies the removal of S3 objects. All other use cases require a user to sign in except for the public pages (landing, about, terms, privacy, contact).

5.2 Architecture and Technology Stack

This is a decoupled web system, with a Next.js 16 front end that runs on Vercel and a tenant that communicates with it via Firebase auth. The core five-step document upload pipeline is powered by a Dockerized FastAPI Backend running on Render, with authenticated requests handled by the Dockerized Backend. This active pipeline verifies file size and type, extracts text using pdfplumber and Tesseract OCR, compares the lease to an RTB model agreement, redactes the contents of the PII using an Irish-specific PII regex and spaCy, securely stores redacted PII in AWS S3, and logs the SHA-256 hashes in a MySQL database. The larger system includes a notice checker, timeline management, and Gemini API input for clause inquiries, and an "in progress" analysis pipeline is being developed to split and classify individual clauses to provide plain-English legal verdicts.

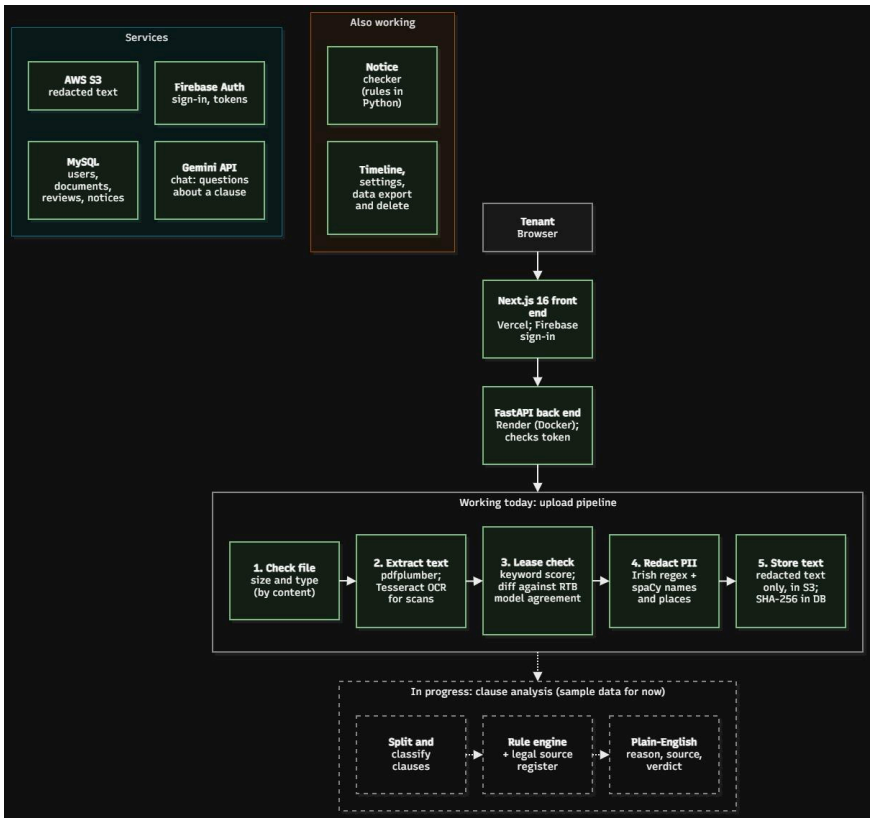


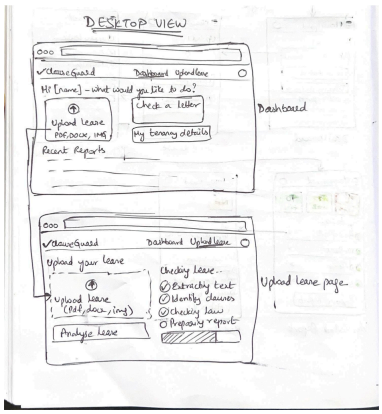
Figure 11: *ClauseGuard architecture as built on 22 October 2026.*

Table 10: *Technology stack and the reason for each choice.*

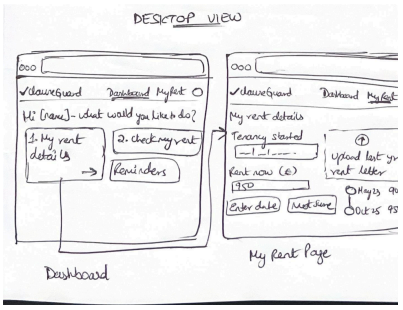
Layer	Technology	Why
Front end	Next.js 16.3, React 19.2, TypeScript 5, Tailwind CSS 4, lucide-react icons, Fraunces + Figtree fonts	Server and client components, file-based routing, strong typing of the API contract (`lib/types.ts`); Tailwind tokens give one consistent design system.
Identity	Firebase Authentication (web SDK + Admin SDK 6.5)	Secure email/password sign-in and password reset without storing passwords; ID tokens verified on every API call.
API	Python 3.11, FastAPI 0.115, Pydantic 2.11, Uvicorn	Async, typed request/response models, automatic OpenAPI docs (Swagger, ReDoc) used for Postman testing.
Persistence	SQLAlchemy 2.0 (async, aiomysql), Alembic 1.13, MySQL 8	Mentor recommended a managed SQL service; migrations keep every environment on the same schema. (Moved from PostgreSQL to MySQL in Sprint 2.)
Documents	pdfplumber 0.11, pytesseract + Tesseract, pdf2image + Poppler, Pillow, python-magic	Text PDFs, scanned PDFs and photos; MIME type is sniffed from the bytes, not trusted from the browser.
Privacy	Regular expressions + spaCy 3.8 (`en_core_web_sm`)	Structured identifiers (PPS, IBAN, Eircode) are pattern-based; names and places need NER.
Analysis	Own rule engine (PyYAML), clause splitter, keyword baseline; PyTorch/Hugging Face for experiments	Deterministic, testable decisions; rules reviewable by non-programmers.
Language model	Google Gemini via `google-genai` (temperature 0.2)	Low-cost, fast per-clause answers in plain text.
Storage	AWS S3 (boto3), private bucket	Redacted text only, per-user prefixes, lifecycle rules for automatic expiry.
Operations	Docker, Render (Frankfurt), Vercel, Google Cloud SQL, slowapi, structlog	Free or low-cost tiers; Docker needed for system packages (Tesseract, Poppler, libmagic).

5.3 Design Process: From Sketches to the Built Interface

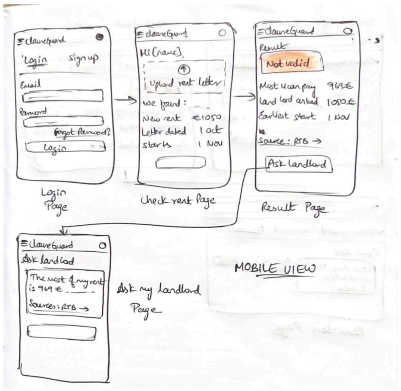
The interface was designed in four iterations: hand-drawn **low-fidelity prototypes** (desktop and mobile), greyscale **wireframes** of the six key screens, **medium-fidelity prototypes** in Figma, and finally the **built interface**. Each iteration changed in response to feedback. The clearest example is the result screen: the early sketches showed a list of highlighted clauses; after supervisor feedback that highlighting alone is not enough, the medium-fidelity design added counts (“1 needs attention, 5 check, 10 fine”), and the built version leads with a verdict sentence and an issue-by-issue walkthrough.



Desktop: landing, dashboard, upload

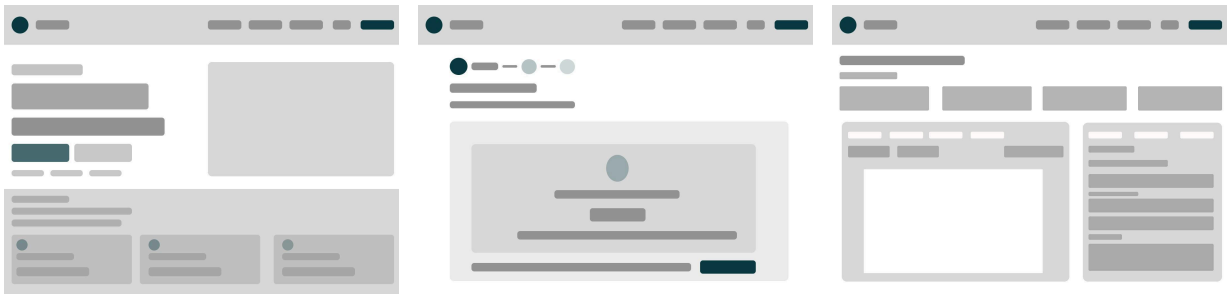


Desktop: colour-coded report, my rent page



Mobile: upload → checking → result

Figure 12: *Low-fidelity paper prototypes (Sprint 2).*



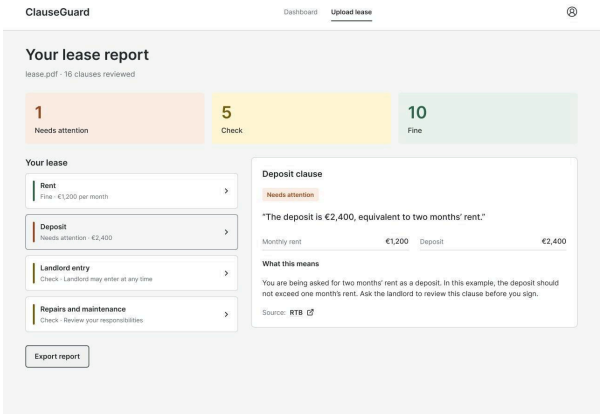
Landing *Upload lease* *Lease review*

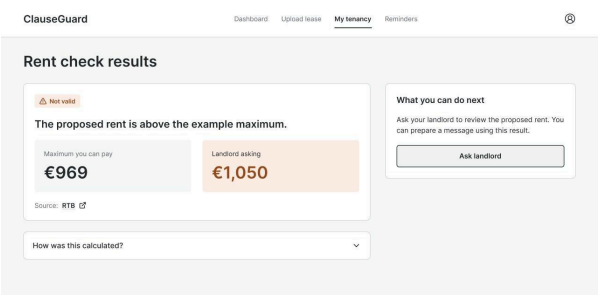
Figure 13: *Desktop wireframes of three of the six key screens.*



Upload *Analysing* *Lease review*

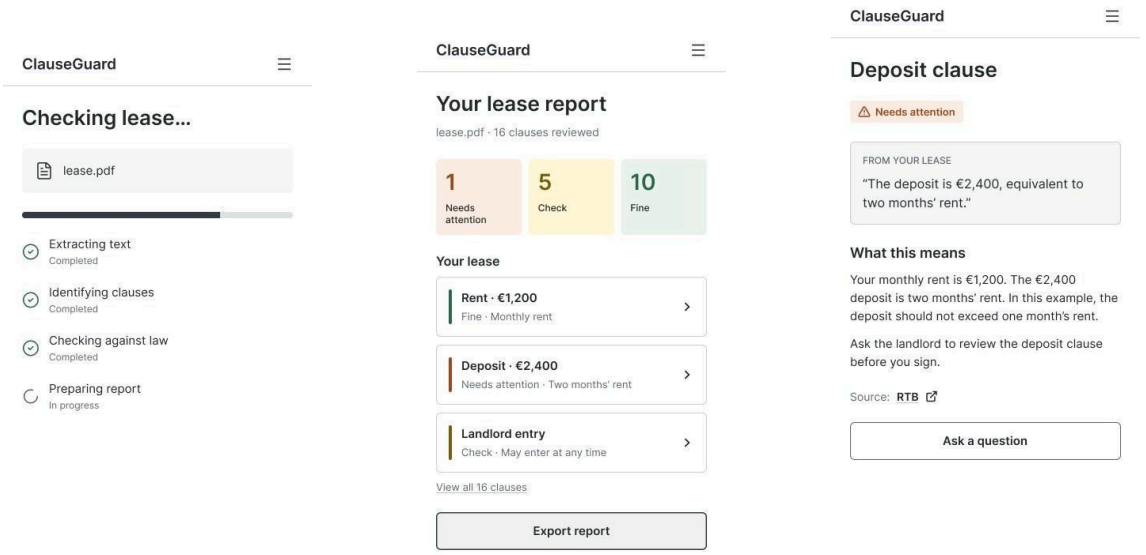
Figure 14: *Mobile wireframes, designed alongside desktop because one persona uses only a phone.*





Colour-coded lease report with clause detail *Rent check result (maximum lawful rent)*

Figure 15: *Medium-fidelity prototypes (desktop).*



Checking lease (progress) *Your lease report* *Deposit clause detail*

Figure 16: *Medium-fidelity prototypes (mobile).*

Figure 18 is the conceptual target for the result screen that we presented after the 16 October feedback session. It adds **topic traffic lights** (rent, deposit, entry and privacy, repairs, term, fees) above the clause list, so a tenant can see at a glance which areas of the lease are a problem. The verdict banner, colour-coded clauses and explanation panel from this concept are built.

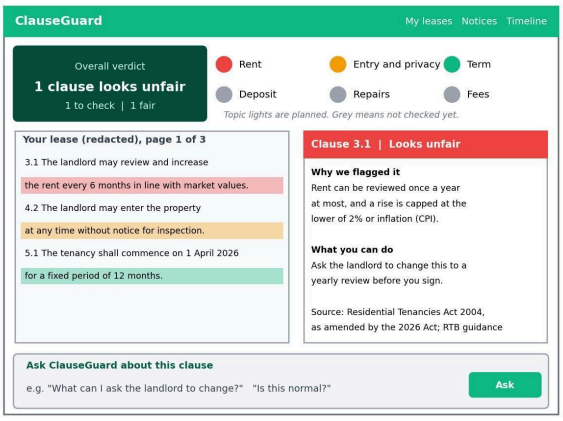


Figure 17: *Conceptual mock-up of the target result screen (illustrative content; topic traffic lights not yet built).*

5.4 Front End

The front end is a Next.js 16 application with three route groups: **public** pages (landing, about, terms, privacy, contact), **auth** pages (login, register) that redirect signed-in users, and **protected** pages wrapped in a `RequireAuth` guard (dashboard, leases, new lease, lease review, notices, notice detail, timeline, settings). A typed API client (`lib/api.ts`) attaches the Firebase ID token to every request, applies timeouts (90 s, or 180 s for uploads to allow for OCR), re-creates a missing profile once, and signs the user out if the account is deactivated. The production build compiles and type-checks cleanly (15 pages and 2 API routes).

Table 11: *Front-end pages and what each one does.*

Page	Purpose and key features
Landing, About, Terms, Privacy, Contact	Explain the product in three steps (upload, check, understand), the four features, the disclaimer and the privacy approach. “Take the tour” is available without an account.
Login / Register	Two-panel design (redesigned after feedback that the first version was too complicated). Registration requires 8+ characters, an upper-case letter and a number, and a matching confirmation; the button stays disabled until the rule is met. Forgot-password flow via Firebase.
Dashboard	Start here: check a lease, check a notice, open the timeline, recent analyses; replay the tour.
Check a lease	Drag-and-drop or file picker; client-side type and 25 MB checks; processing card with steps; clear errors (for example 422 “not a lease”).
Check a notice	Choose notice type (termination, rent increase, entry, other), answer guided questions with “not sure” where appropriate, see verdict, checks, key dates and next steps; list of previous checks.
Timeline	Upcoming / past / all events, rent summary, add events of type event, deadline or rent, tick done, delete.
Settings	Simple or detailed explanations, reminders on/off, GDPR data export (JSON), delete all analyses, delete account.

Accessibility and comprehension features follow the persona insights: plain-English copy throughout, colour never used alone (every status also has an icon and a label such as “worth checking”), a **guided tour** on each main page (`lib/tours.ts`: seven tours covering home, dashboard, leases, upload, review, timeline and notices, remembered per browser), a confidence note when the classifier is unsure (“ClauseGuard is not very confident about this result – double-check it”), the legal disclaimer on every analysis page, and a responsive layout that works at phone width.

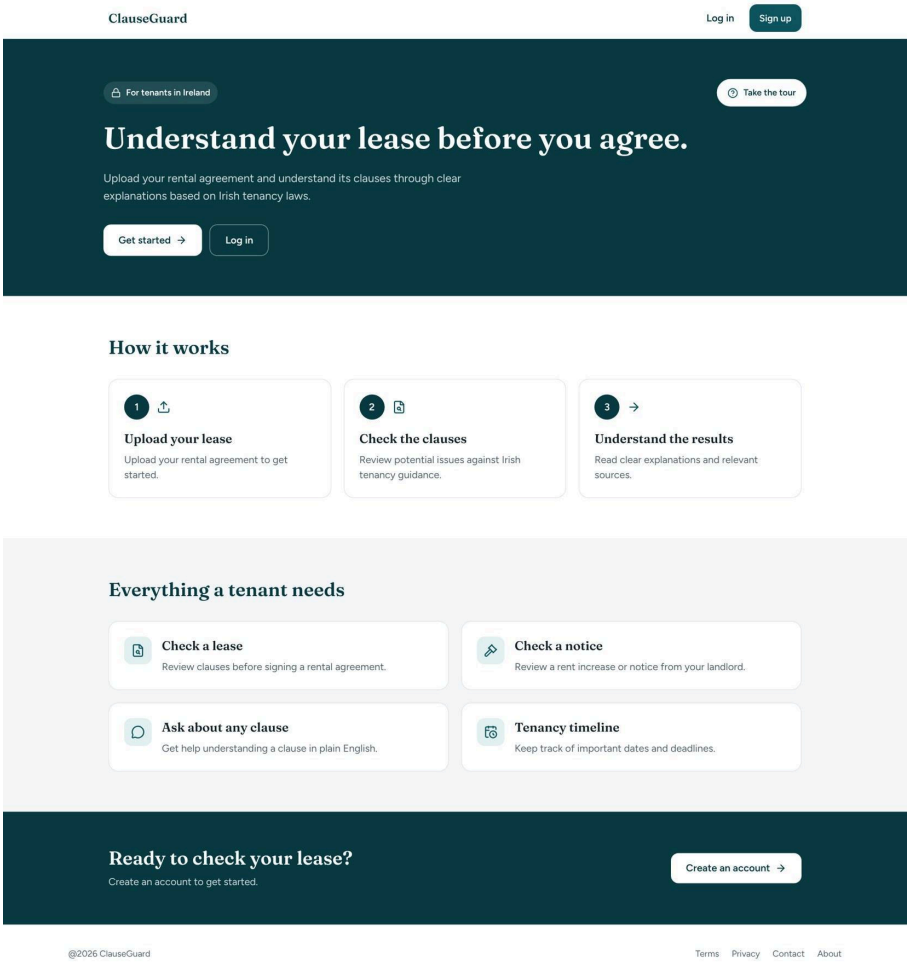
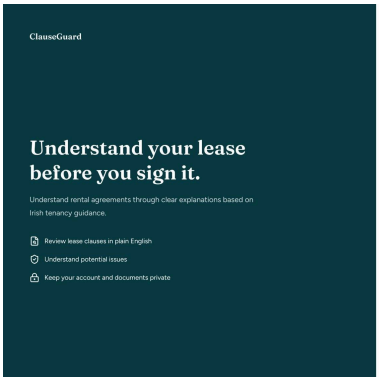
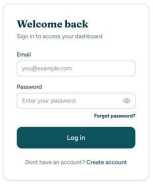
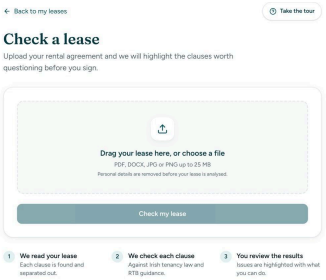


Figure 18: *Built landing page (production build, desktop).*







Login (redesigned two-panel layout) *Check a lease (upload)*

Figure 19: *Built sign-in and upload screens.*

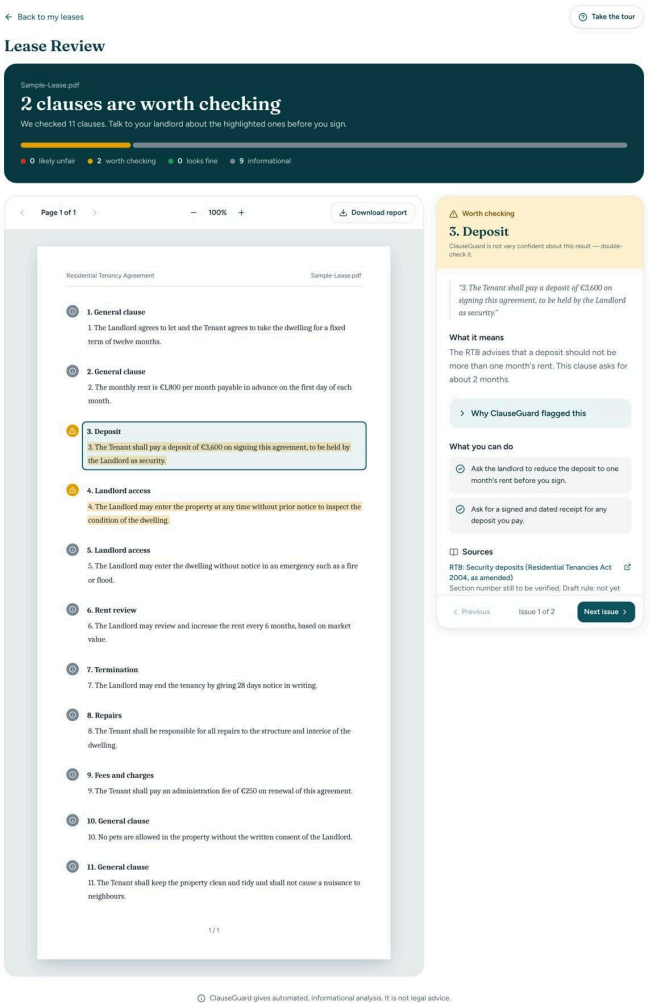
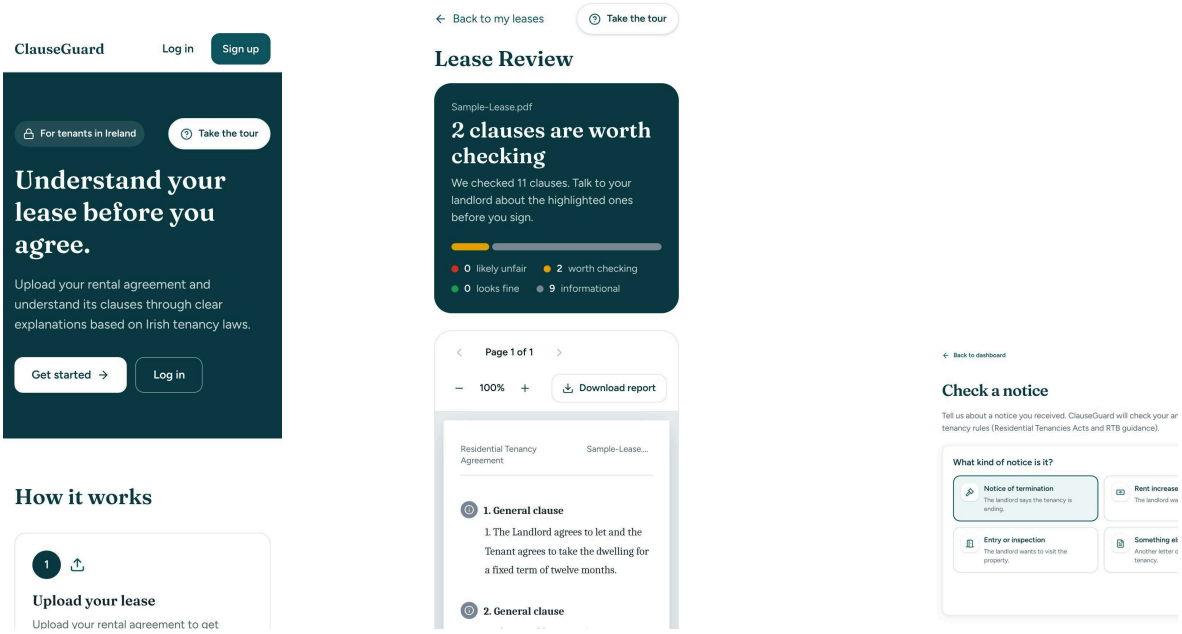


Figure 2: *Post Review Output of the Lease Agreement Pointing out the issues.*



Landing (phone) *Review (phone)* *Check a notice: choose the notice type*

Figure 21: *Mobile layouts and the notice checker.*

5.5 Back End

5.5.1 API, authentication and security

All product endpoints sit under one `/api` base path. Every protected request carries a Firebase ID token in the `Authorization: Bearer` header, which the back end verifies with the Firebase Admin SDK before loading the user; every query is then filtered by that user's ID, so a user can never read another user's documents (ownership is checked even when the client supplies a valid analysis ID). Other controls: an explicit CORS allow-list (no wildcards in production), MIME type sniffed from the file bytes with libmagic, a 25 MB size cap, rate limits with slowapi (60 requests per minute by default, 10 per minute on authentication routes), structured request logging with structlog, an audit log for account events, and UUID primary keys.

Table 12: *API endpoints (21 product endpoints, plus `/health` and two development-only token routes).*

Method	Path	Purpose
POST	/api/auth/register	Create the MySQL profile linked to the Firebase UID; audit event
POST	/api/auth/logout · /api/auth/forgot-password	Revoke tokens; send a Firebase password-reset email
GET	/api/analyses	List the user's analyses with red/amber/green/grey counts
POST	/api/analyses	Upload a lease → extract → validate → redact → S3 → analyse → store; returns the analysis
GET	/api/analyses/{id}	Full clause-by-clause review (camelCase, matches the front-end `Analysis` type)
PATCH	/api/analyses/{id}/clauses/{clause_id}	Mark a clause as reviewed
POST	/api/analyses/{id}/clauses/{clause_id}/questions	Ask ClauseGuard about one clause
DELETE	/api/analyses	Delete all analyses and purge S3 objects
GET / POST	/api/notices · /api/notices/{id}	List, run and read notice checks
GET / POST / PATCH / DELETE	/api/events · /api/events/{id}	Timeline events
GET / PATCH	/api/me/preferences	Explanation detail and reminders
GET / DELETE	/api/me · /api/me/gdpr-export	Profile; cascading account wipe; GDPR Article 15 export

5.5.2 Document pipeline and lease validation

The upload handler reads the file, rejects it if it is larger than 25 MB or if the sniffed MIME type is not an allowed type, computes the **SHA-256 hash of the original bytes**, creates a `documents` row in state *processing*, and calls `run_upload_pipeline()`. The pipeline extracts the text (Section 4.4), then asks the **TemplateDiffEngine** whether the text is a residential lease. The engine scores tenancy vocabulary and requires the parties (landlord and tenant) or tenancy-plus-rent terms; anything else is rejected with a clear message (*"Uploaded file does not appear to be a residential lease agreement"*). For accepted leases, the engine also compares the text with an RTB-style model agreement using Python's `difflib` and lists omissions, inserted clauses and modified standard wording; these deviations are logged today

and will be surfaced to the tenant as “missing standard protections” later. The pipeline then redacts the text, uploads the redacted text to S3 and returns it, in memory only, to the analysis step.

5.5.3 Personal-data redaction

Redaction (`redactor.py`) runs in two passes. The first applies Irish-specific regular expressions: IBAN (`IE` + 20 characters, with or without spaces), PPS number (7 digits + 1–2 letters), Irish mobile and landline numbers (+353 or 0 prefix), email addresses, Eircodes and Dublin postal districts, and passport numbers. The second runs spaCy named-entity recognition and replaces PERSON entities with `[REDACTED_NAME]` and GPE, LOC and FAC entities with `[REDACTED_ADDRESS]`. If the NER model is missing, redaction fails closed: the upload is stopped rather than stored unredacted. Section 6.2 shows our test of this component, which found that it removes all the personal data in our sample but also over-redacts (for example the word “Landlord”), which is the main fix for the next sprint.

5.5.5 Rule engine

The rule engine (`rules_engine/`) is the core of ClauseGuard's “deterministic first” design. It checks one clause, given its topic, the classifier's confidence and optional lease facts, and returns a verdict, the rule that fired, the basis (law or guidance), a plain-English explanation with the extracted numbers filled in, next steps, a citation and the law version. **Rules are data, not code**: they live in `rules/rules.yaml` so that people who do not program, including a solicitor, can read and review them. Each rule has an ID, topic, status (draft or verified), basis, optional `applies_from` / `applies_to` tenancy dates, a check type with parameters, explanation templates for fail-law, fail-guidance, ok and unsure, next steps, and a citation with `verified_on` and `verified_by` fields.

Table 13: *Rule-engine verdicts and how the interface presents them.*

Verdict	Basis	Colour	Meaning shown to the tenant
breaks_law	law	Red	The clause is not allowed.
against_guidance	guidance	Amber	The RTB advises against it, but it is not a legal rule. The difference must be said in words, not only in colour.
needs_review	any	Grey	We are not sure. No guess is made (low confidence, value not found, no verified rule, or start date unknown).
looks_fine	any	Green	Within the limit we checked.

Adding a rule follows a written process that separates the author from the checker: copy an entry and keep it as draft; cite the Act section and RTB page; **someone other than the author** writes test cases in `tests/cases.yaml`, with expected verdicts taken from the law, not from the rule; a reviewer checks the rule against the statute and fills in `verified_on` and `verified_by`; and `pytest` must pass before merging. By default the engine *ignores* draft rules; the demo runs with `include_drafts=True` and labels every result as draft in the interface. The engine's known limits are documented: number extraction is English-only and takes the first match near the key word, so a phrase like “first month's rent plus a deposit of two months” can confuse it, in which case the safe answer is needs review.

5.5.7 Ask ClauseGuard

Ask ClauseGuard answers a tenant's question about one clause. After checking that the clause belongs to the signed-in user, the endpoint sends Gemini a fixed system instruction (an assistant for Irish residential tenancy law, plain text, concise, actionable), the conversation so far, and a prompt containing

only the clause title, the clause text and the question, never the whole lease, with temperature 0.2. This keeps personal data exposure and prompt-injection surface small. Its limitations are clear: answers are not yet grounded in retrieved legal text, there is no enforced "I don't know", and if the model call fails the endpoint currently returns a generic fallback paragraph that refers to rent pressure zones; this fallback must be replaced by an honest error message. Grounding answers in the legal register via retrieval (FR-12, FR-23) is the planned fix.

5.6 Model Experiments

The NLP stream (led by Shubham) investigates whether a learned model can classify clause topics better than keywords, and whether pre-training on Irish law helps. The project plan's architecture uses pretrained sentence-transformer embeddings and a reranker for retrieval, a Legal-BERT classifier fine-tuned on our labelled clauses, the rule engine for every legal decision, and conformal calibration on a held-out set to decide when to answer "needs review". Two experiments have been completed (Figure 24).

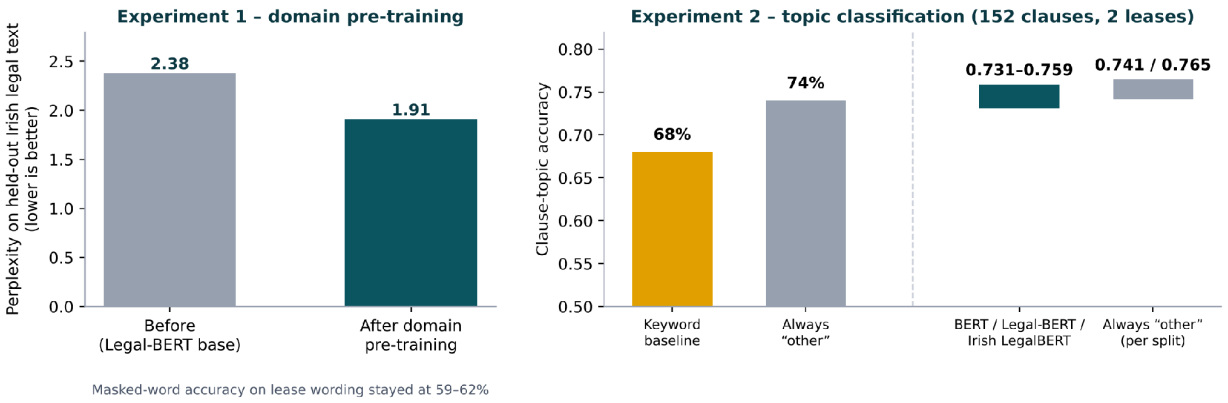


Figure 22: *Results of the two model experiments. Left: domain pre-training lowered perplexity on held-out Irish legal text. Right: on 152 labelled clauses no model beats the majority-class ("other") baseline.*

Experiment 1 – domain pre-training. Legal-BERT was further pre-trained (masked-language modelling) on about 500 Irish Acts, the Residential Tenancies Act 2004 and a 10,000-clause sample of LEDGAR, producing two "Irish LegalBERT" variants. Perplexity on a new held-out set of Irish legal text fell from **2.38 to 1.91**, so the model did learn Irish legal language. However, on a fair comparison using the same hidden words and ten random masks on two leases, masked-word accuracy on *lease* wording stayed at **59–62%** for Legal-BERT and both Irish variants. Training on statute text helped on statutes, but not on the language of leases.

Experiment 2 – clause-topic classification. With 152 clauses from two leases labelled by topic, models were trained on one lease and tested on the other (a lease-level split). BERT, Legal-BERT and Irish LegalBERT scored **0.731–0.759** accuracy, while always answering "other" scored **0.741 and 0.765** on the two splits; the keyword baseline scored **68%** against 74% for always "other". No model was a clear winner and none beat the majority class. With so few labels the models cannot be told apart, and accuracy is a misleading metric when "other" dominates; per-class precision, recall and F1, and the false-green rate, are the metrics that matter.

5.7 Deployment and DevOps

The front end is deployed on **Vercel** (an initial deployment was made on 8 October). The back end is packaged with a production **Dockerfile** (Python 3.11-slim with Tesseract, Poppler and libmagic installed and the spaCy model baked into the image) because Render's native Python runtime lacks these system packages. On start-up the container runs `alembic upgrade head` and then Uvicorn on the port Render assigns. The database is **MySQL 8 on Google Cloud SQL** (europe-west1, SSL-only connections) and redacted text is stored in **S3** (eu-north-1, all public access blocked, an IAM policy limited to put/get/delete on the bucket). Firebase credentials are mounted as a secret file. A written deployment guide and post-deployment checklist (health check, Swagger, CORS, migration, upload and non-lease rejection, notice checker, Q&A, auth flow) are in the repository.

Back-end deployment errors were being fixed during the week of this report, and a free-tier cold start of 30–60 seconds (loading spaCy and checking migrations) is a known risk for demonstrations; the team will either warm the service before presenting or use the low-cost paid tier. Cost is managed by using free tiers and credits wherever possible; S3 lifecycle rules and bucket costs were researched in Sprint 3.

Version control follows a simple rule agreed on 13 October: all commits go to a **stage** branch, and stage is merged to **main every Wednesday** once tested. A continuous-integration pipeline that runs the test suite on every pull request is planned (it was part of the original Definition of Done but is not yet set up).

5.8 Testing and Quality

The back end has **18 automated tests**, all passing on 22 October (run with pytest, using in-memory SQLite and mocked S3, Firebase and libmagic so they run anywhere):

- **Rule engine (14 tests):** eleven expected-verdict cases (T01–T11) covering deposits by months and by euro amount, unknown rent, missing amount, low classifier confidence, entry without notice, entry with notice, an uncovered topic (pets) and the unfilled termination table; plus tests that drafts are ignored by default, that a guidance-basis rule gives amber not red, and that the law version is chosen by tenancy start date (and that an unknown start date gives needs review).
- **Analysis service (3 tests):** deposit and entry flagged amber; an unknown topic is grey, not green; emergency entry is not flagged.
- **API (1 test):** `POST /api/analyses` followed by `GET /api/analyses/{id}` returns the real analysis, not placeholder data.

The rule tests embody an important practice: test cases are written from the law, by someone other than the rule author, so that tests check the rule rather than restate it. Gaps remain: there are no automated tests yet for extraction, redaction, the notice checker or the front end, and no CI pipeline. Section 6.2 reports additional manual tests we ran for this report.

5.9 Key Design Decisions

- **Deterministic first, AI second.** Anything with a numeric or date threshold (deposit, notice period, rent review, rent cap) is a written rule with a source, so the answer is repeatable and explainable.

Language models are reserved for understanding ambiguous wording and for answering questions.

- **Honest uncertainty.** “Needs review” is a first-class outcome. Unassessed clauses are grey, never green. The interface states when the classifier is not confident and when a rule is still a draft.
- **Law versus guidance.** A clause that breaks the law (red) is distinguished from one that goes against RTB advice (amber), in words as well as colour.
- **Rules as reviewable data.** YAML rules with citations, verification fields and independent test cases can be checked by a solicitor who does not read Python.
- **Point-in-time by design.** Rules carry the tenancy dates they apply to, and the register maps which rule applies when.
- **Privacy by design.** Redact before storing; never store the original file; send only one clause to the language model; full export and deletion; fail closed if redaction cannot run.
- **Plain English for non-lawyers.** Explanations, next steps and sources first; detailed legal text optional.
- **Low cost and simple operations.** One back-end service, managed databases, free tiers, a single Dockerfile.

6. Evaluation

6.1 Success Criteria

ClauseGuard succeeds if an ordinary tenant understands their lease better after using it, trusts what it tells them, and what it tells them is correct. We define success on six dimensions (Table 16). Following supervisor advice, accuracy targets for the classifier will be set from the research literature on contract and lease review (literature review in progress) rather than chosen by us; the minimum bar is to beat the majority-class and keyword baselines on per-class F1 with confidence intervals.

Table 16: *Success criteria, measures and targets.*

Dimension	Measure	Target
Correctness	Per-class precision, recall and F1 with bootstrapped 95% confidence intervals on a frozen, lease-split test set; false-green rate (violations shown as fine); extraction accuracy for rent, deposit, notice and dates; parse success by document type (PDF, scan, photo, DOCX)	Beat the majority-class and keyword baselines; literature-derived target for F1; false-green rate as close to zero as possible; every flag cites a source
Trust and abstention	Citation validity (cited section exists and supports the result); calibration (expected calibration error); conformal abstention coverage	100% of red/amber flags with a valid source; uncertain cases marked needs review at the chosen coverage level
Comprehension	Comprehension quiz on a lease answered before and after using ClauseGuard; participant explains why a clause was flagged	Clear improvement in correct answers; most participants can explain a flag in their own words
Usability	System Usability Scale (SUS) with at least 10 prospective tenants; task completion and time on task	SUS at or above the benchmark of 68; upload-to-verdict completed without help by most participants; notice check under 10 minutes (NFR-03)
Privacy and robustness	Personal-data recall and precision of redaction on a labelled PII test set; prompt-injection attack success rate; automated test pass rate	No personal data left in the test set; over-redaction low enough not to change meaning; injection success rate reported with the cost of defences
Performance and cost	End-to-end latency per lease by document type; cost per lease	Text PDF verdict within about one minute (project-plan goal); free-tier or low-cost operation

6.2 Evaluation Results So Far

A first round of user evaluation (24 participants) has been completed; results are in Section 6.2.4 For this interim report we therefore ran the technical checks that are possible today on the code in the repository. All runs below used the team's code unchanged, on synthetic documents written for testing (no real tenant data).

6.2.1 Automated tests

Table 17: *Automated test results on 22 October 2026 (pytest; 19 passed, 0 failed).*

Test file	What it covers	Result
rules_engine/tests/test_rules.py	11 expected-verdict cases + drafts ignored + guidance gives amber + law version by start date	Pass (14/14)
tests/test_analysis_service.py	Deposit and entry flagged; unknown topic grey; emergency entry not flagged	Pass (3/3)
tests/test_analysis_persist.py	Analysis written as review, clause and source rows	Pass (1/1)
tests/test_upload_analysis.py	Upload then read returns the real analysis	Pass (1/1)
Front end (`next build`)	TypeScript type-check and production build of all routes	Pass

6.2.2 End-to-end run on a synthetic lease

We wrote an 11-clause synthetic lease that deliberately contains typical problems (a two-month deposit, entry at any time, six-monthly rent reviews, a 28-day termination notice, all repairs on the tenant, a renewal fee) alongside normal clauses, and ran it through the analysis service exactly as the upload endpoint does. The output is the screen shown in Figure 21; Table 18 lists every clause.

Table 18: *Clause-by-clause output of the analysis service on the synthetic test lease (rent €1,800, deposit €3,600).*

#	Clause (abridged)	Topic shown	Result	Assessment of the result
1	Fixed term of twelve months	General clause	Grey (no rule)	Correct to abstain
2	Rent €1,800 per month in advance	General clause	Grey (no rule)	Acceptable; rent extracted as a lease fact
3	Deposit of €3,600 on signing	Deposit	Amber	Correct: about 2 months against a 1-month guidance limit
4	Landlord may enter at any time without prior notice	Landlord access	Amber	Correct: entry without notice
5	Landlord may enter without notice in an emergency	Landlord access	Grey (needs review)	Correct: emergency entry not flagged
6	Rent reviewed every 6 months	Rent review	Grey (no rule)	Missed: should be red (P-01); rule not yet written
7	Landlord may end the tenancy on 28 days' notice	Termination	Grey (needs review)	Correct to abstain (start date unknown; notice table not filled); bug: explanation shows a raw “{reason}” placeholder
8	Tenant responsible for all repairs to the structure	Repairs	Grey (no rule)	Missed: should be flagged; no repairs rule yet
9	Administration fee of €250 on renewal	Fees and charges	Grey (no rule)	Missed: should be flagged; no fees rule yet
10	No pets without written consent	General clause	Grey (no rule)	Correct to abstain
11	Keep the property clean, no nuisance	General clause	Grey (no rule)	Correct

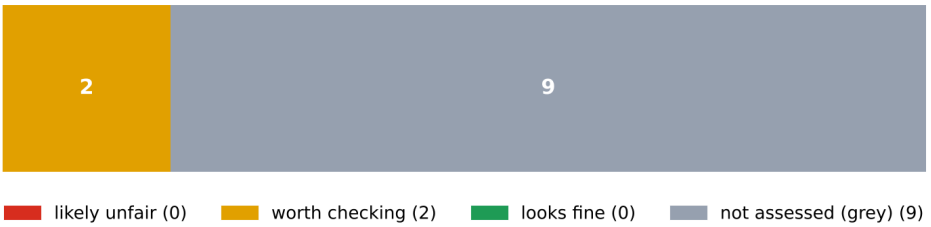


Figure 23: Result counts for the synthetic lease: 2 worth checking, 9 not assessed, none wrongly marked fine.

Findings. (1) The splitter found all 11 clauses and the lease facts (rent €1,800, deposit €3,600) were extracted correctly. (2) Every flag that was raised was correct, and **no clause was wrongly shown as fine** (false-green rate 0 on this lease), which is the most important safety property. (3) Three clauses that a tenant should be warned about (rent review, repairs, fees) were not assessed because the rules do not exist yet; this is recall, not precision, and it is the main development priority. (4) We found a presentation bug: when a date-bound rule cannot run, the unsure template is filled with an empty value set, so the text "{reason}" appears to the user; the fix is to pass the reason into the template. (5) The analysis step itself takes about **0.8 ms per lease** in our test environment (mean of 50 runs), so end-to-end time will be dominated by OCR, redaction and network, which we will measure on deployment.

6.2.3 Redaction test

We tested the redactor on a synthetic paragraph containing one of each kind of personal data, using the same spaCy model as the production image. Table 19 shows the result.

Table 19: Redaction test on synthetic personal data (spaCy `en_core_web_sm`).

Input	Output	Result	Comment
John Murphy / Pavan Kumar (names)	[REDACTED_NAME]	Pass	Both names removed
PPS 1234567TA	[REDACTED_PPS]	Pass	
087 123 4567; john.murphy@example.ie	[REDACTED_PHONE]; [REDACTED_EMAIL]	Pass	
IBAN IE29 AIBK 9311 5212 3456 78	[REDACTED_IBAN]	Pass	
14 Main Street, Dublin 7, D07 X2Y3	14 [REDACTED_ADDRESS], [REDACTED_ADDRESS], [REDACTED_ADDRESS]	Partial	House number left; acceptable but to tighten
"the Landlord", "The Landlord may enter…"	"the [REDACTED_ADDRESS]", "The [REDACTED_NAME] may enter…"	Fail	Over-redaction of the party role changes meaning
RESIDENTIAL TENANCY AGREEMENT (title)	[REDACTED_NAME]	Fail	Over-redaction
Reference number AB1234	[REDACTED_PASSPORT]	Fail	Passport pattern too broad
€1,800; 1 March 2026; 28 days	unchanged	Pass	Numbers needed by the rules survive

All personal data in the sample was removed (recall 100% on this small test), and the numbers the rule engine needs survived. But the NER model frequently tags the capitalised words "Landlord" and "Tenant" as names or places, and the passport pattern matches ordinary reference codes. On our

11-clause lease the verdicts did not change after redaction (still two amber), because the rules key on words such as “deposit” and “enter”; but the clause text shown to the tenant was damaged (“The [REDACTED_ADDRESS] may enter the property…”), and party roles matter for future rules. **Fix planned:** an allow-list of legal role words (Landlord, Tenant, Agent, Licensee, Dwelling, Premises), restricting the passport pattern to the Irish format and labelled contexts, and a labelled PII test set to measure precision and recall properly.

6.2.4 User Evaluation

A first round of user evaluation has been completed with 24 participants. It used a task-based usability questionnaire (Google Form) that followed the user journey: landing page, sign-up and login, dashboard, upload, the analysis wait screen, the lease review screen and next steps. Most questions were 1–5 ratings or yes/no/partly answers, with free-text questions on where participants got stuck and what one change would help. The mean rating across the six core steps was 4.0 out of 5. Upload was the strongest step (4.33) and 16 of 24 participants rated it 5. Finding problem clauses (3.79) and switching between the Explanation, Sources and Ask tabs (3.83) were the weakest. The weaker results were mostly about understanding the output rather than operating the interface. Only 14 of 24 participants (58%) fully understood the difference between "Likely unenforceable", "Needs review" and "Compliant", and 9 understood it only partly. Participants who understood the labels rated finding problem clauses 4.6 on average, against 2.6 for those who understood them only partly, so unclear labels appear to reduce the usefulness of the whole review screen. Fifteen of 24 participants knew what to do after reading a flagged clause's explanation. Seven of 24 (29%) said they at some point did not know where they were or how to go back. Of the 15 who named a step where they got stuck, five named the review screen, four named the waiting screen and three named the dashboard. In the free-text answers, participants asked for simpler, less legal wording for the risk categories. They also asked for a back button and a menu with the main actions, clearer accepted file types on the upload page, and an upload option on the landing page. One noted that mobile navigation was harder, and another that long leases took a long time to analyse. These findings have been added to the next sprint backlog: plainer risk labels, clearer navigation, visible upload requirements and better progress information. Round 2 will re-test the changed screens and add the System Usability Scale and a comprehension quiz. This round did not use SUS, so its results are not compared with the benchmark of 68. The sample is small and self-reported, and the landing-page and login questions were answered by only five early participants.

Results.

Figure 24 summarises the results. The mean rating across the six core steps was 4.0 out of 5.

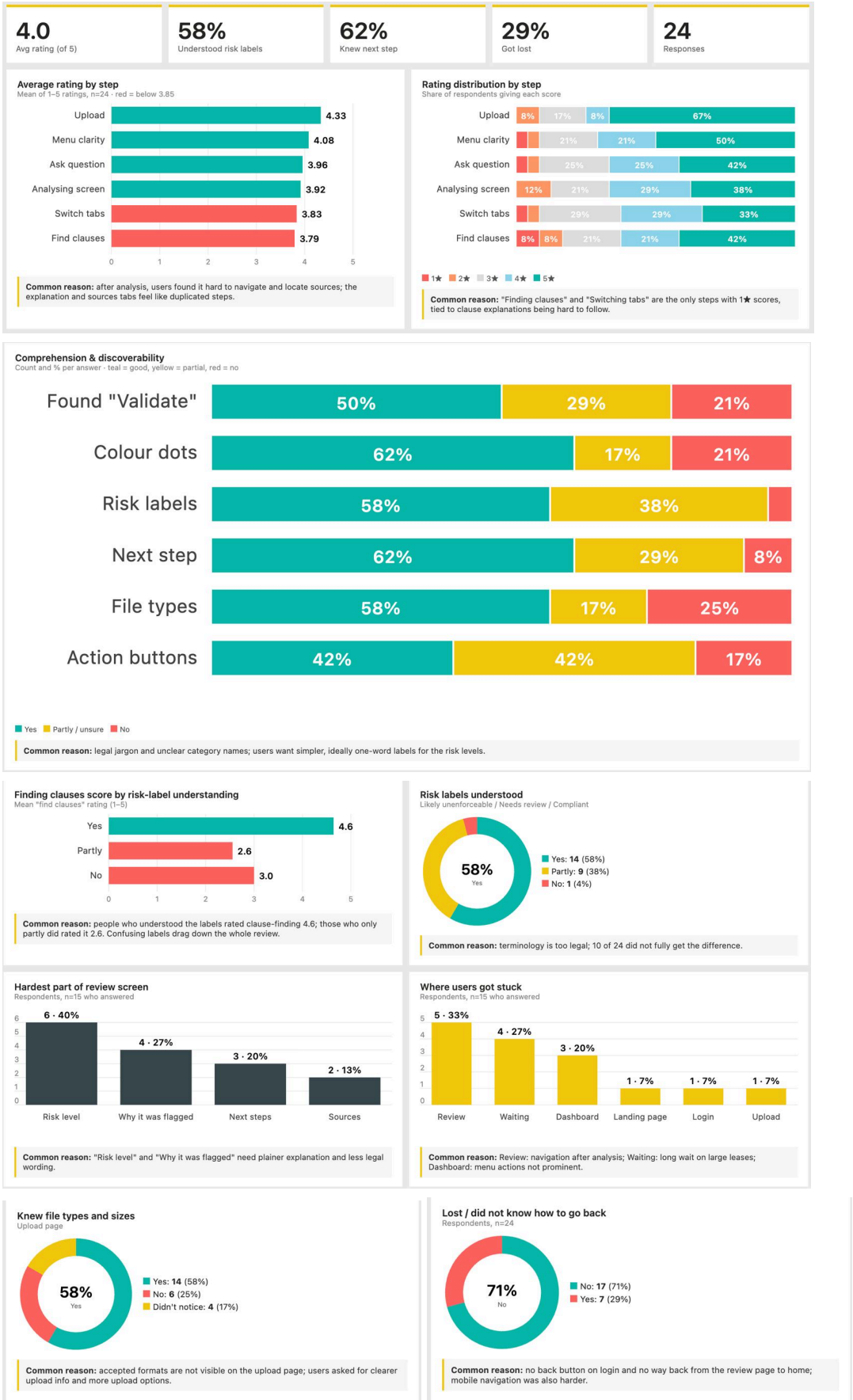


Figure 24: *User inputs after using Clauseguard phase 1.*

6.2.5 Lease validator

The synthetic lease was accepted with a confidence of 0.56 (8 of 20 vocabulary terms matched, including landlord, tenant, tenancy, rent, dwelling and deposit) and 12 deviations from the RTB-style baseline were computed. A short invoice text was rejected with confidence 0.0. The threshold therefore separates obvious cases; it will be tested on real non-lease uploads (notices, IDs, utility bills) during user testing.

6.2.6 Notice checker on Isabel's letter

We entered Isabel's situation from the persona (rent €950 rising to €1,050, letter giving 30 days, no comparable rents, last review 12 months ago). The checker returned **invalid** with these checks:

Table 20: *Actual notice-checker output for the Isabel persona scenario.*

Status	Check	Detail and source
Fail	Insufficient notice (30 days vs 90 required)	Fewer than 90 days before the new rent begins. RTA 2004 s. 22(2)
Fail	Increase exceeds the cap (10.53% vs 2.0%)	The requested increase exceeds the 2% limit. RTA 2004 s. 24B
Pass	Review frequency (12 months since last review)	RTA 2004 s. 20
Fail	Missing three comparable properties	A rent review notice must cite three comparable dwellings. RTA 2004 s. 22

Next steps returned: refer a dispute to the RTB before the new rent date (within 28 days of receiving the notice) and keep paying the existing rent until it is resolved. The arithmetic and the verdict are right for Isabel, and the output answers her persona goals. The wording of the cap check ("RPZ / HICP") and its section references are exactly the items the legal review must update to the March 2026 national cap.

6.3 Technical Evaluation Methodology

The main technical evaluation is a comparison of **four approaches to clause analysis** on one frozen test set, so that no result is tuned to the data it is evaluated on:

1. **Keyword / rule baseline** – the current production stand-in: keyword topic classification plus the rule engine.
2. **Zero-shot language model** – a general LLM asked to classify each clause and give a verdict, with no task-specific training.
3. **Fine-tuned small model** – Legal-BERT (or the Irish variant) fine-tuned on our labelled clauses for topic classification, feeding the rule engine.
4. **Hybrid retrieval-augmented system** – rules for numeric checks; for other clauses, retrieve the relevant provisions from the versioned knowledge base (BM25 plus dense retrieval with reranking) and let the model judge against the retrieved text, with citation verification and conformal abstention.

Table 21: Technical evaluation metrics and how each will be computed.

Metric	Definition and procedure
Per-class P / R / F1	For each of the nine topics and for the verdict labels; bootstrapped 95% confidence intervals (1,000 resamples of leases).
False-green rate	Share of clauses labelled non-compliant that the system shows as fine. Treated as the primary safety metric.
Abstention coverage	Share of clauses answered (not needs review) at a fixed error rate, with the threshold set by conformal prediction on a calibration split.
Calibration	Expected calibration error of the model's confidence.
Citation validity	Share of flags whose cited section exists in the register and supports the verdict (checked by a second person).
Extraction accuracy	Exact match for rent, deposit, notice lengths and dates; error propagation measured as verdicts changed by extraction errors.
Point-in-time accuracy	On a date-sensitive subset (tenancies before and after 1 March 2026), share of clauses judged against the right version of the law.
Parsing success	Share of documents parsed into clauses correctly, by type (text PDF, scanned PDF, photo, DOCX).
Injection robustness	Attack success rate on leases containing hidden text or injected instructions; accuracy and latency cost of the defences.
Latency and cost	Seconds per lease and cost per lease for each approach.

The test set will be frozen on **15 November**, with a target of at least 500 labelled clauses split by lease, inter-annotator agreement measured with Cohen's kappa on a double-labelled subset, and an active-learning labelling tool compared with random sampling for labelling efficiency (a stretch objective). A category with weak recall will receive more labelled data or a rule-based fallback in the following sprint.

6.4 User Evaluation Methodology

User evaluation will test the product with **ordinary non-lawyers**, starting with international students (closest to the Pavan persona), followed by workers and older tenants. The questionnaire was confirmed and the Google Form created on 17 October. Figure 26 shows the protocol for each session.

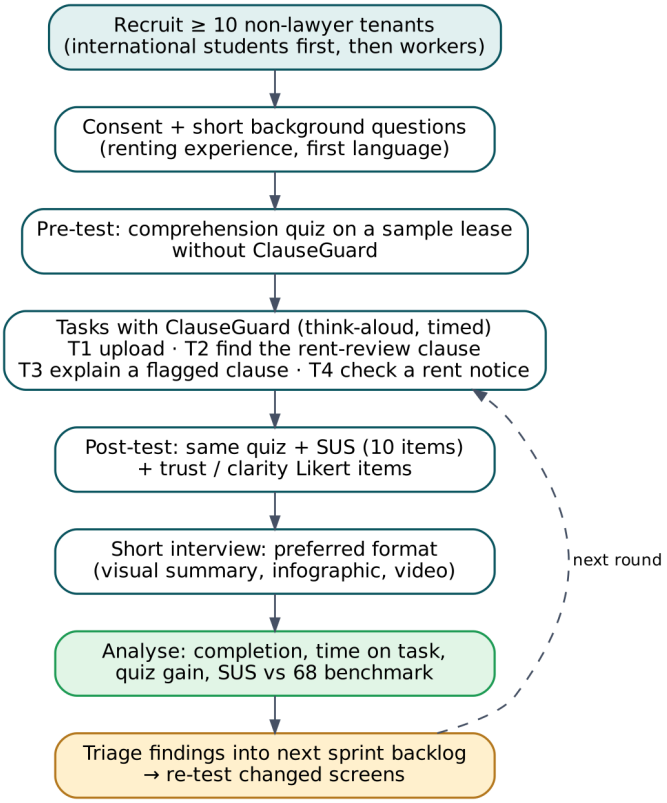


Figure 25: *User study protocol. Each round's findings are triaged into the next sprint and the changed screens are re-tested.*

- **Participants:** at least 10 prospective or current tenants without legal training, recruited through student networks and personal contacts; informed consent; no real personal data required (a sample lease is provided).
- **Tasks:** T1 upload the sample lease and read the verdict; T2 find the rent-review clause and say whether it is acceptable; T3 explain in your own words why a flagged clause was flagged and what you would do; T4 answer “how much would it cost me to leave early?”; T5 check a rent-increase letter with the notice checker.
- **Measures:** task completion and time on task; comprehension quiz before and after; SUS (10 items, scored 0–100, compared with 68); Likert items on trust and clarity; observed errors and confusion (think-aloud).
- **Format preferences:** we will ask how participants would like a lease explained (visual summary, infographic, short video), as suggested by our supervisors.

Our supervisors stressed that prototypes are only useful when tested with users. We will therefore test the current interface now, record the feedback in the typed feedback log, change the design and re-test, in two rounds (late October and mid-November), rather than leave evaluation to the end.

6.5 Legal Correctness Check

Legal correctness is evaluated separately from usability. A solicitor will be asked to review the **rules and their sources**, not the user interface. The review will start with the items the register and the rule file flag: rules marked draft, sections marked “TO VERIFY”, register entries marked open or partial (P-13, P-14, P-25, P-26), and the notice-checker wording that still refers to Rent Pressure Zones. A rule

becomes *verified* only when a second person has checked it against the statute and the RTB page and signed it off in the rule file.

6.6 How Evaluation Feeds Development

Evaluation is designed to change what we build, sprint by sprint: a topic with weak recall gets more labels or a rule fallback; a rule with a wrong citation is downgraded to draft until fixed; usability friction found in a session goes straight into the next sprint backlog; and any redaction miss blocks real-user testing until it is fixed and covered by a test. The findings in Section 6.2 have already produced four backlog items: rent-review, repairs and fees rules; the placeholder bug; the redaction allow-list; and the page-number fix in the extractor.

7. Conclusion: The Plan

7.1 Project Management Strategy

The project is run with **Scrum** over the twelve teaching weeks from 21 September to 13 December 2026, planned as six two-week sprints with fixed external milestones from the project plan: proposal and repository sign-off by 9 October; architecture, schemas and CI gates by 18 October; test set frozen on 15 November; full evaluation results by 29 November; feature freeze on 30 November; final report, demo and submission on 13 December. In practice, because the module has a presentation every week, the first five weeks ran as **one-week cycles**, each with updates, a class presentation and (from 4 October) a retrospective, grouped in the repository as Pre-sprint and Sprints 1–4.

- **Tools:** GitHub is the system of record for code, issues and branches; the sprint board started on Jira (set up on 20-30 September) and tasks are tracked against it; retrospectives use Retrium; meetings use Microsoft Teams (earlier Google Meet).
- **Meetings:** frequent short stand-ups (25 team meetings logged in five weeks), mentor meetings roughly every two weeks, sprint reviews in the weekly class presentation, and a written retrospective at the end of each sprint.
- **Documentation:** every team and mentor meeting is logged (date, attendees, platform, discussion, decisions, tasks) in `Project_Management/Meetings/`; since the 16 October feedback session we also keep a **typed feedback log** of each supervisor comment and how it was addressed.
- **Branching:** commits go to a `stage` branch and are merged to `main` every Wednesday once tested (agreed 13 October).
- **Decision-making and conflict** (from the project plan): consensus first, otherwise the role owner decides within their area; cross-cutting decisions by majority; issues not resolved within a sprint go to the retrospective as an action with an owner and date, then to the project manager, then to the supervisor. On legal-content disputes, whoever completed the RTB cross-check for that item decides, because that decision is documented and checkable.

7.2 Team Roles and Contributions

Roles were assigned by strength in the project plan and refined on 10 October, after lecture advice, into a clear front-end / back-end / model split. Below it summarises each member's role and main contributions to date, taken from the meeting logs and weekly presentations.

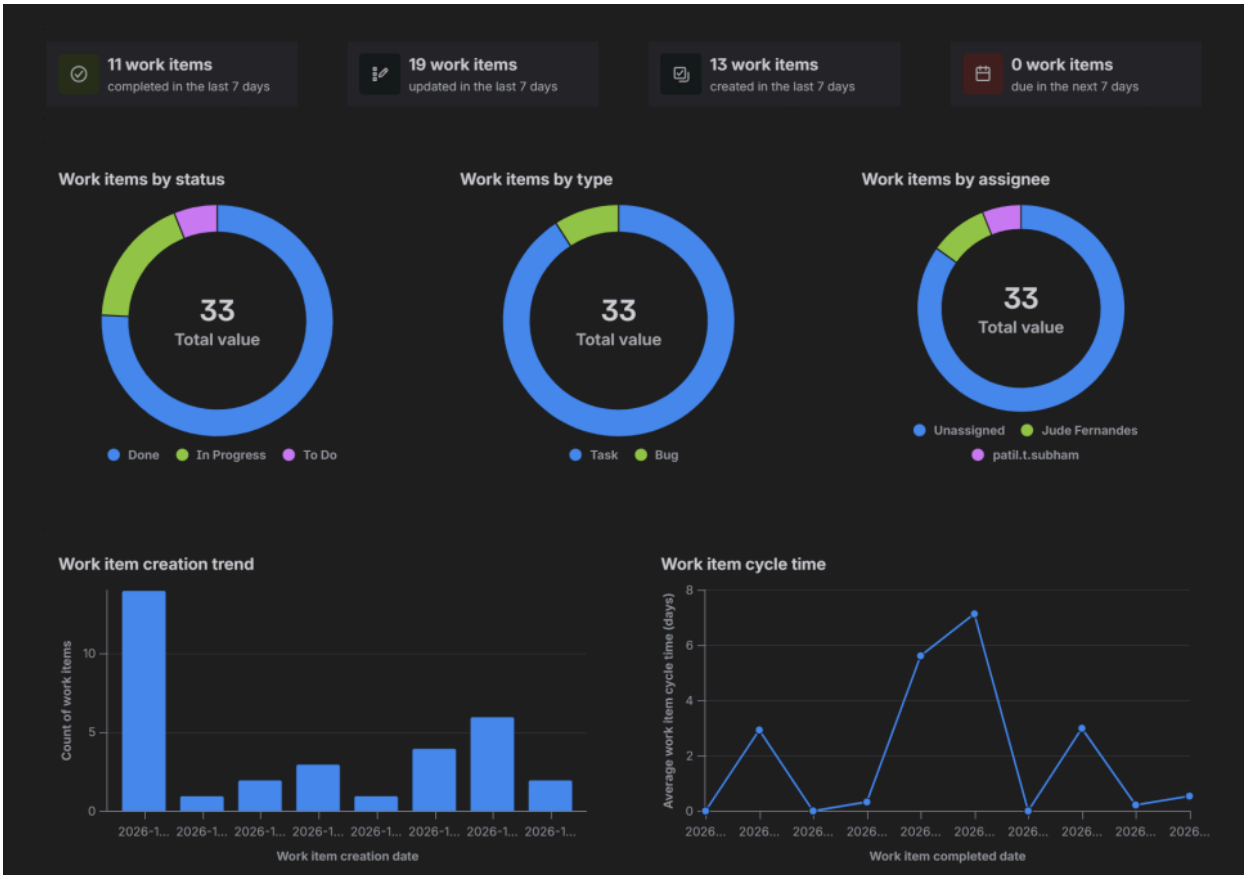
Table 22: *Team roles and main contributions to date.*

Member	Role	Main contributions so far
Siddharth Varma Raja (A00047237)	Project Manager / Legal Research Lead; back end	Sprint board and repository structure; all meeting logs and retrospectives; RTB template diff engine and lease validator; PII redaction (with Prasad); S3 lifecycle and cost research; integration and deployment fixes.
Prasad Nitin Pansare (A00047190)	Back-end and Knowledge-Base Engineer	FastAPI service; Firebase authentication and token verification; migration from PostgreSQL to MySQL; SQLAlchemy models and Alembic migrations; document upload, OCR extractor and pipeline; S3 integration; API documentation and Postman guide; deployment set-up.

Member	Role	Main contributions so far
Shubham Tatyasaheb Patil (A00047193)	NLP / AI Engineer	Legal model research (LegalBERT); domain pre-training of Irish LegalBERT; clause splitter, labelling tool and lease-level splits; 152 labelled clauses; keyword baseline and model comparison; rule engine and its tests.
Kameshwara Karthik Ayyalasomayajula (A00046404)	Front-end / UX Developer	Personas, mind map and competitor analysis (with Jude); low-fidelity prototypes, wireframes, medium-fidelity and Figma mock-ups; initial interactive mock-up (10 October); Next.js front end and Vercel deployment.
Jude Fernandes (A00046791)	QA, Data and Documentation Lead; front end	Personas and user journeys/flows (with Karthik); testing-tool and hosting research; front-end development; user evaluation questionnaire and Google Form (with the team).

7.3 Meetings, Retrospectives and Team Health

Between 18 September and 20 October the team logged **28 meetings** (25 team meetings and 3 mentor meetings). Across them, the five core members recorded **126 of 140 possible attendances (90%)**. Figure 27 shows attendance per meeting: full attendance became the norm from 13 October, after two thinly attended meetings on 10 and 11 October were raised in the Sprint 3 retrospective.



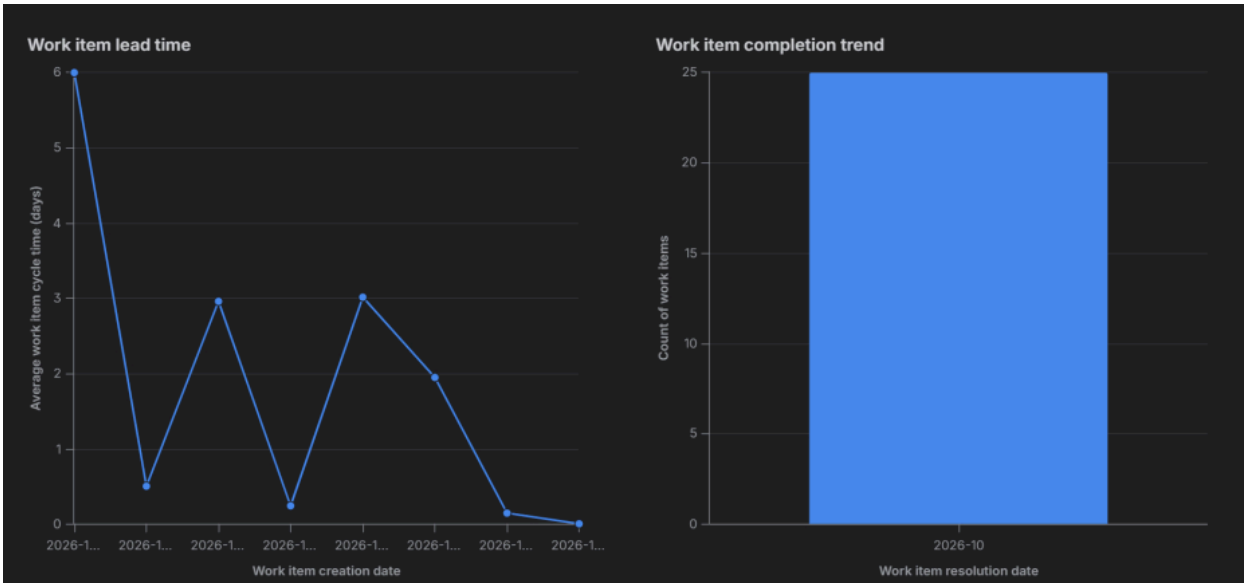


Figure 26: *Meeting and Attendance.*

7.4 Progress to Date

Table 24: *Progress against the plan by area (22 October 2026).*

Area	Status	Detail
Concept and plan	Done	Idea confirmed after mentor review (30 Sep); project plan, requirement specification, personas, mind map, competitor analysis, journeys and flows complete; repository structure agreed.
Legal knowledge base	Partial	Legal source register v2 (18 instruments, 27 provisions, point-in-time map, change log); 23 of 27 provisions RTB-verified; 4 open or partial; solicitor review not yet held.
Front end	Done	All 15 pages built and type-checked: public pages, auth, dashboard, upload, review workspace, notice checker, timeline, settings, guided tours; deployed to Vercel.
Back end core	Done	Auth, upload checks, SHA-256, extraction with OCR, lease validator, redaction, S3, notice checker, timeline, preferences, GDPR export and deletion, rate limits, audit log, 4 migrations.
Clause analysis	Partial	Real end-to-end analysis replaces the earlier sample data: splitter, keyword classifier, rule engine (3 draft rules) and persistence. Missing: rent-review, repairs, fees and TMD rules; retrieval.
Model	Partial	Two experiments; 152 labelled clauses; no model yet beats the baseline; more labelled data needed.
Testing	Partial	19 automated tests passing; no CI yet; no tests for extraction, redaction or the front end.
Deployment	In progress	Front end live on Vercel; back end Docker image and guide ready; deployment errors being fixed.
User evaluation	Planned	Questionnaire and Google Form ready; first round week of 26 Oct.

7.5 Response to Supervisor Feedback

Table 25: *Supervisor feedback and how we have responded.*

Feedback	Our response
Stay with the idea but improve it significantly (30 Sep)	Kept ClauseGuard; added the notice checker, timeline, point-in-time legal register and the deterministic rule engine as the differentiators.
Test with real, ordinary people, not lawyers	Target users redefined; lawyers only check the rules. Questionnaire ready; first round with international students planned for the week of 26 Oct.
Plain English, not legal citations	Explanation panel leads with “what it means”, “why it was flagged” and “what you can do”; sources listed separately; simple/detailed setting.
Go beyond highlighting: verdict and dashboard	Verdict banner and counts built; issue-by-issue navigation; topic traffic lights designed (Figure 18) and planned; visualisation literature to be reviewed.
Show green for good clauses	Green “looks fine” status exists; grey is used for “not assessed” so we never imply a clause is fine when we have not checked it.
Let tenants ask questions about the lease	Ask ClauseGuard works per clause; questions on costs, maintenance and the cost of leaving early to be added; grounding planned.
Get something working on a real lease	A lease now runs through extraction, validation, redaction, splitting, rule checks and storage to the review screen. Real clause flags exist for deposit and entry; more rules are the main remaining step.
Do not pick an accuracy target yourselves	Literature review on lease/contract interpretation benchmarks added to the plan; baselines and per-class F1 with confidence intervals defined.
Keep a visible, typed list of feedback	Typed feedback log started after 16 Oct; this table is drawn from it.
Simplify the login; fix the password check	Login and registration redesigned (two-panel layout); registration enforces 8+ characters, an upper-case letter and a number.

7.6 Current Challenges and Risks

ID	Risk	Mitigation and owner
R1	Clause analysis stays rule-thin (few clauses assessed)	Write rent-review (P-01/P-02), repairs (P-14), fees and TMD termination rules first; move notice-checker logic into the shared rule file. Owner: NLP and back end.
R2	Too few labelled clauses to evaluate models	Second annotator; more template leases; synthetic and adversarial clauses; labelling sessions scheduled each week; target 500 by 15 Nov.
R3	Redaction damages text (over-redaction) or misses data	Role-word allow-list; tighter passport pattern; labelled PII test set with precision/recall; block real-user tests until fixed.
R4	A legal rule is wrong or out of date	Draft-until-verified process; solicitor review of rules; monthly re-check of the register; law version returned with every result; disclaimer on every page.
R5	Integration and deployment failures	Deployment guide and checklist; staged merges on Wednesdays; CI to run tests on every pull request; warm the service before demos.
R6	Too few user-study participants	Recruit through student networks early; short sessions; remote option; sample lease provided so no personal data is needed.
R7	Prompt injection through an uploaded lease	Only one clause sent to the model; fixed system instruction; hidden-text detection and an injection test set planned.
R8	Attendance and workload balance	Visible task board; retrospective actions with owners; everyone contributes to code and documentation.
R9	Cloud cost and free-tier cold starts	Free tiers and credits; S3 lifecycle expiry; consider the low-cost paid tier for presentation days.

Other technical issues found while preparing this report are already in the backlog: DOCX files are accepted at upload but not yet extracted; page numbers are lost because pages are joined with newlines; the unsure-template placeholder bug; the Ask ClauseGuard fallback text; and the notice checker's pre-2026 rent-cap wording.

7.7 Timeline and Deliverables

Below shows completed and planned work against the project-plan milestones, and Table 27 lists the focus for each remaining period. The priority order is deliberate: first make the core lease check genuinely useful on real leases (rules), then collect the evidence (labels, user studies), then compare approaches, then harden and write up.

Table 27: *Plan for the remaining weeks, aligned with the project-plan deadlines.*

Period	Focus and deliverables
To 23 Oct (interim)	Submit this report and the presentation; demo upload → review, notice checker; finish back-end deployment; fix the placeholder bug and page numbers.
Week of 26 Oct	First user evaluation round (international students); literature review on accuracy benchmarks, simplification and legal visualisation; redaction allow-list and PII test set; rent-review and fees rules.
By 1 Nov	First end-to-end slice on real leases with real flags and citations for deposit, entry, rent review, fees and termination (pre- and post-1 March 2026); CI running the test suite.
By 15 Nov	500+ labelled clauses with a second annotator and Cohen's kappa; test set frozen; topic traffic lights; DOCX extraction; second user evaluation round with SUS.
By 29–30 Nov	Four-way comparison (keyword, zero-shot, fine-tuned, hybrid RAG) with per-class F1, false-green rate and abstention; solicitor check of rules; feature freeze.
To 13 Dec	Hardening, injection tests, performance and cost measurement, DPIA; final report, demo and submission.

7.8 Summary

Five weeks in, ClauseGuard has moved from a proposal to a working system: a tenant can sign in, upload a lease, have it validated, redacted and analysed, and read a colour-coded, plain-English review with next steps and sources, as well as check landlord notices and track their tenancy. The architecture reflects the lessons of the research and of our supervisors' feedback: deterministic, reviewable rules make the legal decisions; uncertainty is shown, never hidden; the law is versioned by date; and personal data is removed before anything is stored. Our own tests show both what works (no wrongly-green clauses, correct notice verdicts, all personal data removed) and what does not yet (too few rules, over-redaction, too little labelled data). The remaining seven weeks are planned around closing exactly those gaps and producing the evidence, from labelled data and from real tenants, that ClauseGuard helps.

8. References and Key Resources

[1] Angelopoulos, A. N., & Bates, S. (2021). *A gentle introduction to conformal prediction and distribution-free uncertainty quantification.* arXiv:2107.07511. https://arxiv.org/abs/2107.07511

[2] Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human–Computer Interaction, 24*(6), 574–594. https://doi.org/10.1080/10447310802205776

[3] Brooke, J. (1996). SUS: A “quick and dirty” usability scale. In P. W. Jordan et al. (Eds.), *Usability Evaluation in Industry* (pp. 189–194). Taylor & Francis.

[4] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020). LEGAL-BERT: The Muppets straight out of law school. In *Findings of EMNLP 2020* (pp. 2898–2904). https://doi.org/10.18653/v1/2020.findings-emnlp.261

[5] Citizens Information. (2026). *Renting a home.* https://www.citizensinformation.ie/en/housing/renting-a-home/

[6] Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37–46. https://doi.org/10.1177/001316446002000104

[7] ClauseGuard Group 3. (2026). *RTB-compliant legal source register, v2* (compiled 22 September, updated 9 October 2026). Internal project document.

[8] ClauseGuard Group 3. (2026). *Project plan; requirement specification; meeting logs; retrospectives; weekly presentations (Weeks 1–4).* Internal project documents, project repository.

[9] Dahl, M., Magesh, V., Suzgun, M., & Ho, D. E. (2024). Large legal fictions: Profiling legal hallucinations in large language models. *Journal of Legal Analysis, 16*(1). https://doi.org/10.1093/jla/laae003

[10] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186). https://doi.org/10.18653/v1/N19-1423

[11] European Parliament and Council. (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation).* https://eur-lex.europa.eu/eli/reg/2016/679/oj

[12] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (pp. 79–90). https://doi.org/10.1145/3605764.3623985

[13] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 1321–1330). PMLR.

[14] Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). CUAD: An expert-annotated NLP dataset for legal contract review. In *NeurIPS 2021 Datasets and Benchmarks Track.* https://arxiv.org/abs/2103.06268

[15] Houses of the Oireachtas. (2004–2026). *Residential Tenancies Act 2004* (as amended), *Residential Tenancies (Miscellaneous Provisions) Act 2026*, and *Housing and Residential Tenancies (Miscellaneous Provisions) Act 2026.* Irish Statute Book. https://www.irishstatutebook.ie

[16] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for

knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33* (pp. 9459–9474).

[17] Lippi, M., Pałka, P., Contissa, G., Lagioia, F., Micklitz, H.-W., Sartor, G., & Torroni, P. (2019). CLAUDETTE: An automated detector of potentially unfair clauses in online terms of service. *Artificial Intelligence and Law, 27*(2), 117–139. https://doi.org/10.1007/s10506-019-09243-2

[18] Magesh, V., Surani, F., Dahl, M., Suzgun, M., Manning, C. D., & Ho, D. E. (2025). Hallucination-free? Assessing the reliability of leading AI legal research tools. *Journal of Empirical Legal Studies.* https://doi.org/10.1111/jels.12413

[19] Minister for Housing, Local Government and Heritage. (2019). *Housing (Standards for Rented Houses) Regulations 2019* (S.I. No. 137 of 2019). Irish Statute Book.

[20] Residential Tenancies Board. (2026). *Rental law changes from 1 March 2026.* https://rtb.ie/renting/rental-law-changes-from-1-march-2/

[21] Residential Tenancies Board. (2026). *Important changes to rental law from 14 September 2026.* https://rtb.ie/about/news/important-changes-to-rental-law-from-14-september-2026/

[22] Residential Tenancies Board. (2026). *Security deposits.* https://rtb.ie/renting/rights-responsibilities/security-deposits/

[23] Sauro, J. (2011). *A practical guide to the System Usability Scale: Background, benchmarks and best practices.* Measuring Usability LLC.

[24] Smith, R. (2007). An overview of the Tesseract OCR engine. In *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)* (Vol. 2, pp. 629–633). IEEE. https://doi.org/10.1109/ICDAR.2007.4376991

[25] Threshold. (2026). *Advice for renters.* https://www.threshold.ie

[26] Tuggener, D., von Däniken, P., Peetz, T., & Cieliebak, M. (2020). LEDGAR: A large-scale multi-label corpus for text classification of legal provisions in contracts. In *Proceedings of LREC 2020* (pp. 1235–1241).

[27] Software documentation: Next.js, React, Tailwind CSS, Firebase Authentication, FastAPI, Pydantic, SQLAlchemy, Alembic, slowapi, structlog, pdfplumber, Tesseract/pytesseract, pdf2image, Pillow, python-magic, spaCy, Hugging Face Transformers, Amazon S3, Google Cloud SQL, Render, Vercel, Google Gemini API (official documentation, accessed September–October 2026).